

МИНИСТЕРСТВО ТРАНСПОРТА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ УНИВЕРСИТЕТ ТРАНСПОРТА»
РУТ (МИИТ)
Институт управления и цифровых технологий
Кафедра «Цифровые технологии управления транспортными
процессами»

Курсовая работа

на тему:

«Распознавание дорожных знаков»

по дисциплине

«Системы искусственного интеллекта»

Выполнил:

студент группы УИС-311

Васильев Арсений Денисович

Проверил:

доцент, к.т.н.

Корепанова Вероника Сергеевна

Москва 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 МАТЕМАТИЧЕСКАЯ МОДЕЛЬ	6
1.1 Основы сверточных нейронных сетей	6
1.2 Пулинг-слои	7
1.3 Архитектура типичной CNN для классификации изображений	9
1.4 Функция потерь	11
1.5 Оптимизатор Adam	14
1.6 Метрики оценки качества модели	16
1.7 Проблема переобучения и методы борьбы с ней	17
2 ОПИСАНИЕ АЛГОРИТМА	20
3 РЕАЛИЗАЦИЯ	23
3.1 Предобработка данных	23
3.2 Описание модели	28
3.2.1 Общая структура модели	28
3.2.2 Архитектура сверточной части модели	30
3.2.3 Классифицирующая часть модели	32
3.2.4 Преимущества и особенности выбранного подхода	33
3.2.5 Выбор параметров Dropout	34
3.2.6 Выбор параметра скорости обучения	37
3.3 Проверка на корректность	38
3.3.1 Базовая модель	38
3.3.2 Модель без Dropout	41
3.3.3 Модель без аугментации	42
3.3.4 Модель с упрощенной архитектурой	43
3.3.5 Сравнение моделей	44
ЗАКЛЮЧЕНИЕ	46
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	48
ПРИЛОЖЕНИЕ А	50

СПИСОК РИСУНКОВ

Рисунок 1.1 – График функции активации ReLU	7
Рисунок 1.2 – Сравнение работы MaxPooling и AveragePooling.....	9
Рисунок 1.3 – Сравнение графиков Sigmoid и Softmax.....	11
Рисунок 1.4 – График функции Cross-Entropy Loss.....	12
Рисунок 1.5 – Визуализация принципа работы SGD.....	14
Рисунок 1.6 – Иллюстрация недообучения, нормального обучения и переобучения	18
Рисунок 3.1 – Примеры изображений дорожных знаков из датасета GTSRB...	24
Рисунок 3.2 – Распределение изображений по классам датасета GTSRB	25
Рисунок 3.3 – Визуализация методов аугментации.....	27
Рисунок 3.4 – Архитектура сверточной модели CNN	29
Рисунок 3.5 – Визуализация карт признаков на разных уровнях	32
Рисунок 3.6 – Графики истории обучения базовой модели.....	39
Рисунок 3.7 – Матрица ошибок	40
Рисунок 3.8 – Графики модели без Dropout	42
Рисунок 3.9 – Графики модели без аугментации.....	43
Рисунок 3.10 – Графики модели с упрощенной архитектурой	44

СПИСОК ТАБЛИЦ

Таблица 3.1 – Результаты статистического анализа цветовых характеристик.	23
Таблица 3.2 – Результаты тестирования значений Dropout в классифицирующей части.....	34
Таблица 3.3 – Результаты тестирования значений Dropout2d в сверточной части.....	36
Таблица 3.4 – Результаты тестирования значений скорости обучения.....	37
Таблица 3.5 – Сравнение моделей.....	45

ВВЕДЕНИЕ

В современном мире безопасность дорожного движения остаётся одной из наиболее острых проблем. Ежегодно в дорожно-транспортных происшествиях погибает более 1,3 миллиона человек, а десятки миллионов получают травмы. Значительная часть аварий происходит из-за невнимательности водителей и ошибок при распознавании дорожных знаков. Поэтому разработка систем автоматического распознавания дорожных знаков (Traffic Sign Recognition, TSR) является важной и актуальной задачей в области компьютерного зрения и искусственного интеллекта.

Такие системы составляют неотъемлемую часть современных систем помощи водителю (ADAS) и беспилотных автомобилей. Они позволяют в реальном времени определять тип и значение дорожного знака и передавать эту информацию в систему управления транспортным средством, способствуя повышению безопасности на дорогах.

Стремительное развитие технологий глубокого обучения существенно расширило возможности решения подобных задач. Сверточные нейронные сети (Convolutional Neural Networks, CNN) показали высокую эффективность в классификации изображений и стали основным инструментом для распознавания объектов, в том числе дорожных знаков.

Цель курсового проекта заключается в анализе программной реализации и исследовании особенностей сверточной нейронной сети для классификации изображений на примере датасета GTSRB, направленных на выявление наиболее эффективных методов настройки гиперпараметров и архитектурных решений, обеспечивающих высокую точность распознавания дорожных знаков в системах компьютерного зрения.

Для достижения поставленной цели были решены следующие задачи:

- изучить теоретические основы и математическую модель сверточных нейронных сетей;
- рассмотреть архитектуру и принципы работы сверточных нейронных сетей для задач классификации изображений;

- проанализировать датасет GTSRB и выполнить предобработку данных;
- реализовать и настроить модель сверточной нейронной сети для классификации дорожных знаков;
- провести обучение модели и проверить её качество на валидационной и тестовой выборках;
- проанализировать полученные результаты и оценить эффективность разработанной системы.

Объектом исследования является задача автоматической классификации изображений дорожных знаков средствами глубокого обучения.

Предметом исследования выступает архитектура и процесс обучения сверточной нейронной сети для классификации изображений дорожных знаков на датасете GTSRB.

1 МАТЕМАТИЧЕСКАЯ МОДЕЛЬ

1.1 Основы сверточных нейронных сетей

Сверточные нейронные сети (Convolutional Neural Networks, CNN) в настоящее время являются одним из самых эффективных инструментов для обработки изображений. В отличие от обычных полносвязных нейронных сетей, CNN лучше учитывают пространственную структуру изображения благодаря специальному типу слоёв – сверточным.

Свёртка – это основная операция в таких сетях. Она заключается в том, что небольшое окно (ядро или фильтр) «скользит» по изображению и на каждом участке вычисляет сумму поэлементного произведения значений пикселей и весов ядра. Математически операция свёртки для двухмерного случая записывается следующим образом:

$$(I * K)(x, y) = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} I(x + i, y + j) * K(i, j), \quad (1.1)$$

где $I(x + i, y + j)$ – значение пикселя входного изображения в точке со смещением (i, j) относительно текущей позиции ядра (x, y) ;

$K(i, j)$ – значение веса ядра свертки в позиции (i, j) ;

K – ядро свертки размером m на n .

Важными параметрами свёртки являются:

- размер ядра (Kernel size) – обычно 3×3 или 5×5 ;
- шаг (Stride) – на сколько пикселей сдвигается ядро за один шаг;
- заполнение (Padding) – добавление нулей по краям изображения,

чтобы сохранить размер карты признаков после свёртки.

После применения операции свёртки обычно используется функция активации. Её основная задача – ввести нелинейность в модель. Без функции активации вся сеть, независимо от количества слоёв, сводилась бы к простой линейной трансформации, что сильно ограничило бы её возможности [8].

Наиболее популярной функцией активации в современных CNN является ReLU (Rectified Linear Unit):

$$ReLU(x) = \max(0, x), \quad (1.2)$$

где x – входное значение нейрона.

График функции наглядно демонстрирует её ключевое свойство: при отрицательных входных значениях нейрон «выключается» (выход равен нулю), а при положительных – пропускает сигнал без изменений (рисунок 1.1) [8].

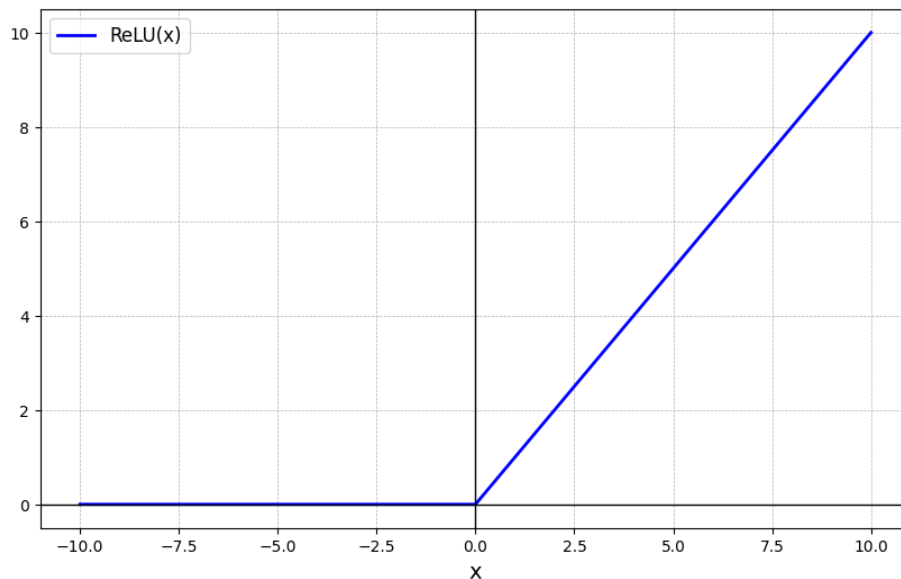


Рисунок 1.1 – График функции активации ReLU

1.2 Пулинг-слои

Важным понятием в сверточных нейронных сетях является карта признаков (feature map). После применения сверточного слоя к входному изображению получается новый массив данных – карта признаков. Каждый элемент этой карты показывает, насколько сильно в определённой области изображения проявляется тот или иной признак (например, вертикальная линия, круг, цветовой переход и т.д.). Чем глубже слой сети, тем более сложные и абстрактные признаки содержат карты признаков.

После сверточных слоёв в нейронной сети обычно применяется пулинг (Pooling). Основная задача пулинга – уменьшить пространственные размеры карты признаков, сохранив при этом наиболее важную информацию [14].

Пулинг решает сразу несколько важных проблем:

- снижает вычислительную сложность сети;
- уменьшает количество параметров;
- делает модель менее чувствительной к небольшим сдвигам, поворотам и искажениям изображения (повышает устойчивость к изменениям).

Самым популярным видом пулинга является MaxPooling. Он делит карту признаков на небольшие непересекающиеся окна (чаще всего 2×2) и в каждом окне оставляет только максимальное значение. Таким образом, после MaxPooling 2×2 размер карты признаков уменьшается в 2 раза по высоте и ширине.

Математически операция MaxPooling записывается следующим образом:

$$P(x, y) = \max_{(i,j) \in R} I(x + i, y + j), \quad (1.3)$$

где $P(x, y)$ – значение карты признаков после пулинга в координатах (x, y) ;

(x, y) – текущие координаты выходной карты признаков;

(i, j) – смещение внутри окна пулинга;

R – окно пулинга;

I – входная карта признаков.

Существует также AveragePooling, который вместо максимума берёт среднее значение всех элементов в окне. AveragePooling иногда используется в последних сверточных блоках, когда нужно получить более «сглаженное» представление признаков. Однако в большинстве современных архитектур для задач классификации MaxPooling показывает лучшие результаты [14].

Ниже представлено наглядное сравнение MaxPooling (рисунок 1.2, а) и AveragePooling (рисунок 1.2, б).

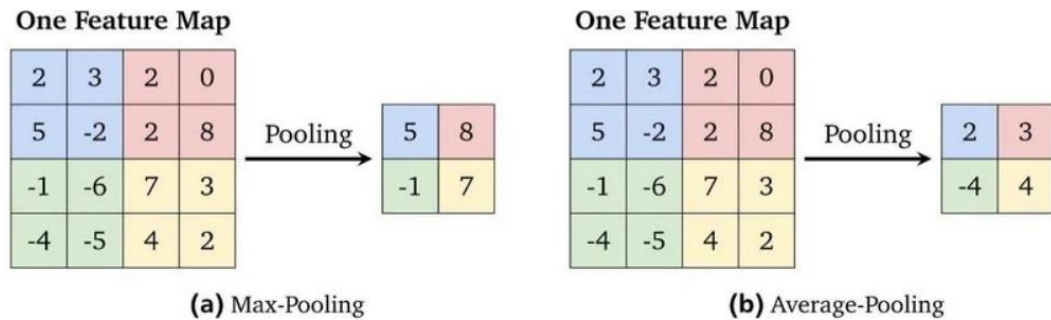


Рисунок 1.2 – Сравнение работы MaxPooling и AveragePooling

В разработанной модели пулинг применяется после каждой группы сверточных слоёв, что позволяет постепенно уменьшать размерность от 32×32 до очень компактного вектора признаков перед полносвязными слоями.

1.3 Архитектура типичной CNN для классификации изображений

Типичная сверточная нейронная сеть для задачи классификации изображений строится по блочному принципу [9]. Каждый блок обычно состоит из следующих слоёв:

- сверточный слой (Convolution) – извлекает локальные признаки (линии, края, текстуры);
- функция активации ReLU – добавляет нелинейность;
- слой пулинга – уменьшает размерность и повышает устойчивость к сдвигам.

Такие блоки повторяются несколько раз. С каждым новым блоком сеть учится выделять всё более сложные и абстрактные признаки: от простых краёв на первых слоях до целых объектов на последних.

После последнего пулинга полученная карта признаков представляет собой многомерный массив (тензор). Чтобы передать его на полносвязные слои, необходимо преобразовать этот многомерный массив в одномерный вектор. Такая операция называется Flatten («выпрямление» или «сглаживание»). После неё карта признаков превращается в обычный вектор

чисел, который уже можно подавать на классифицирующую часть сети – полносвязные слои.

На самом последнем слое используется функция активации Softmax, которая преобразует выходные значения в вероятности классов:

$$Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}, \quad (1.4)$$

где $K = 43$ – количество классов дорожных знаков;

z_i – выходное значение нейрона, соответствующее i -му классу;

j – индекс суммирования по всем классам;

e – основание натурального логарифма.

Ниже представлено сравнение графиков Sigmoid (рисунок 1.3, а) и функции активации Softmax (рисунок 1.3, б). Визуально обе функции имеют характерную S-образную кривую, однако их математическое поведение различается при наличии нескольких выходов. Если Sigmoid преобразует значение каждого нейрона в диапазон $(0, 1)$ независимо от остальных, то Softmax работает со всем вектором выходов сразу, обеспечивая их взаимозависимость. На графике это проявляется в том, что Softmax формирует распределение вероятностей: увеличение значения для одного класса автоматически подавляет значения остальных так, чтобы их общая сумма всегда была равна единице. Это делает Softmax незаменимым для многоклассовой классификации дорожных знаков, так как позволяет четко выделить один доминирующий класс, в то время как сигмоида может выдать высокие значения вероятности сразу для нескольких не связанных между собой категорий.

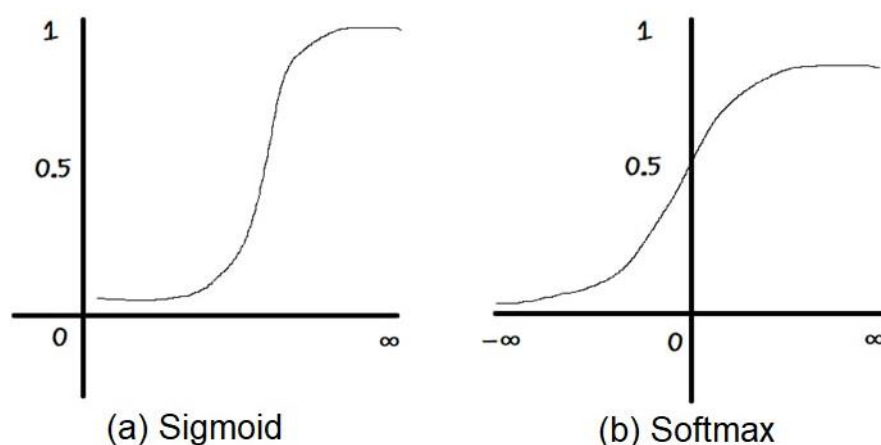


Рисунок 1.3 – Сравнение графиков Sigmoid и Softmax

Такая многоуровневая структура позволяет CNN автоматически извлекать иерархию признаков: от низкоуровневых (границы, цвета) до высокоуровневых (форма и смысл знака). Именно это свойство делает сверточные сети значительно эффективнее традиционных методов компьютерного зрения [8].

1.4 Функция потерь

Для обучения нейронной сети необходимо количественно оценивать, насколько хорошо модель справляется с поставленной задачей. Для этого используется функция потерь (loss function). В задачах многоклассовой классификации, к которым относится распознавание дорожных знаков, наиболее часто применяется кросс-энтропийная функция потерь (CrossEntropyLoss).

Выбор данной функции обусловлен принципом максимального правдоподобия.

Математически удобнее работать не с самими вероятностями, а с их логарифмами. Это превращает произведение вероятностей в сумму их логарифмов, что значительно упрощает вычисление производных (градиентов), необходимых для корректировки весов сети.

Кросс-энтропия измеряет расхождение между истинным распределением классов и распределением, предсказанным моделью. Чем

сильнее предсказанные вероятности отличаются от реальных меток, тем выше значение функции потерь [13].

Математическая формула кросс-энтропии для одного примера выглядит следующим образом:

$$L = - \sum_{i=1}^C y_i \log(\hat{y}_i), \quad (1.5)$$

где L – значение функции потерь;

y_i – истинная метка класса (0 или 1);

$\hat{y}_i$ – предсказанная вероятность класса I ;

C – количество классов (43);

I – индекс класса;

$\log$ – натуральный логарифм.

В данной формуле ключевую роль играет логарифмическая зависимость. Она выполняет функцию «штрафа» для модели. При рассмотрении графика функции логарифма на отрезке от 0 до 1 видно, что при приближении предсказанной вероятности к единице (правильный ответ), значение потерь плавно стремится к нулю. Однако, если предсказанная вероятность правильного класса начинает уменьшаться, значение функции потерь растет не линейно, а логарифмически быстро (рисунок 1.4).

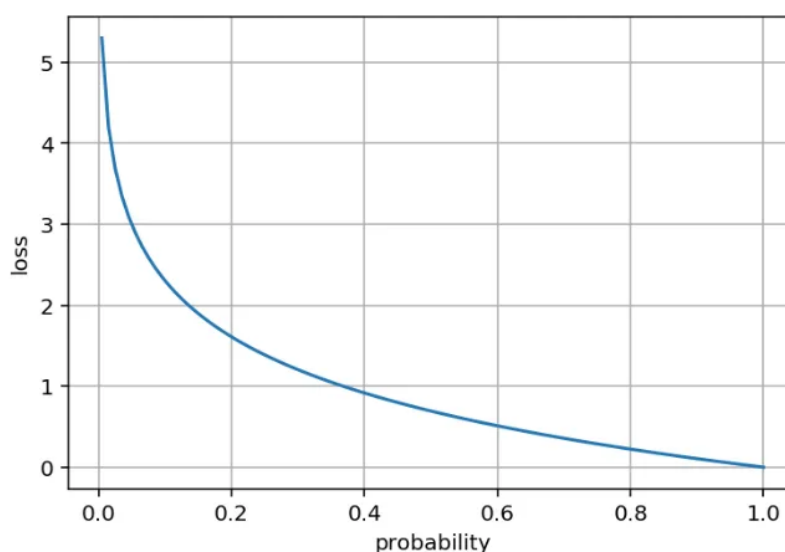


Рисунок 1.4 – График функции Cross-Entropy Loss

На практике обучение нейронных сетей происходит не по одному изображению, а небольшими группами, которые называются мини-батчами. Поэтому формула потерь усредняется по всем примерам в мини-батче:

$$L = -\frac{1}{N} \sum_{k=1}^N \sum_{i=1}^C y_i^{(k)} \log(\hat{y}_i^{(k)}), \quad (1.6)$$

где N – размер мини-батча;

C – количество классов;

k – индекс примера в мини-батче;

i – индекс класса;

$y_i^{(k)}$ – истинная метка класса i для k -го примера;

$\hat{y}_i^{(k)}$ – предсказанная вероятность класса i для k -го примера.

Градиент функции потерь показывает, в какую сторону и насколько сильно нужно изменить каждый вес сети, чтобы уменьшить ошибку. Простота и эффективность вычисления градиентов для CrossEntropyLoss в сочетании с Softmax значительно ускоряют процесс обучения.

Математическое преимущество логарифмической зависимости заключается в высокой чувствительности функции потерь при низких значениях вероятности истинного класса. Поскольку производная функции $-\log(x)$ резко возрастает при приближении аргумента к нулю, это обеспечивает формирование значительного градиента в тех случаях, когда модель допускает грубую ошибку (присваивает верному классу низкую вероятность) [13].

Таким образом, логарифмический характер функции заставляет алгоритм в первую очередь корректировать веса для исправления наиболее критических неточностей. При этом, по мере приближения вероятности к единице, функция становится более полой, что способствует плавной стабилизации весов на финальных этапах обучения.

В ходе работы функция потерь использовалась для оценки качества модели на каждой эпохе. По значению потерь на обучающей и валидационной

выборках можно судить, насколько хорошо сеть обучается и не начинает ли она переобучаться. Снижение значения CrossEntropyLoss в процессе обучения свидетельствует о том, что модель всё лучше различает разные классы дорожных знаков.

1.5 Оптимизатор Adam

Одним из ключевых компонентов обучения нейронной сети является выбор оптимизатора – алгоритма, который обновляет веса модели так, чтобы минимизировать функцию потерь.

Основой для большинства современных методов оптимизации является стохастический градиентный спуск (Stochastic Gradient Descent, SGD). Математически обновление весов в SGD описывается формулой:

$$\omega_{t+1} = \omega_t - \eta * \Delta L(\omega_t), \quad (1.7)$$

где ω_{t+1} – обновленный вектор весов;

η – скорость обучения (learning rate);

$\Delta L(\omega_t)$ – вектор градиента функции потерь;

ω_t – вектор весов модели на итерации t .

Основная проблема классического SGD заключается в использовании фиксированного шага для всех параметров и сильных колебаниях на пути к минимуму, особенно на сложных поверхностях функции потерь (рисунок 1.5).

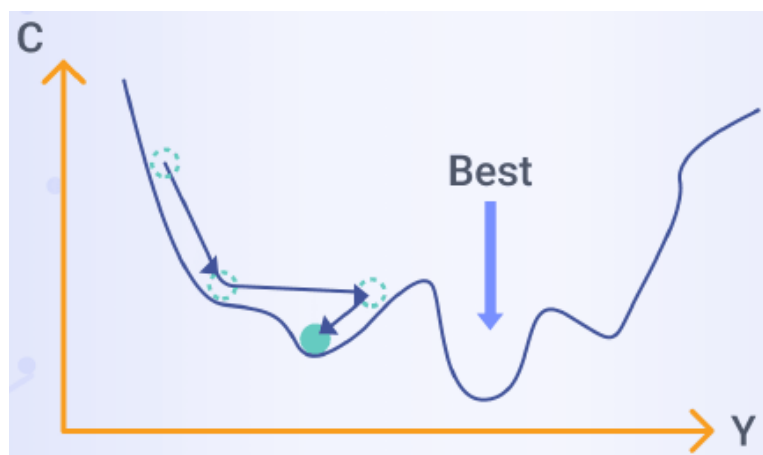


Рисунок 1.5 – Визуализация принципа работы SGD

В данной работе использовался оптимизатор Adam (Adaptive Moment Estimation). Он считается одним из самых эффективных и популярных оптимизаторов для сверточных нейронных сетей [2].

Adam объединяет в себе преимущества двух других методов:

- Momentum – учитывает предыдущие градиенты, чтобы ускорять движение в нужном направлении и сглаживать колебания;
- RMSProp – адаптивно изменяет скорость обучения для каждого параметра отдельно (большие градиенты получают меньшую скорость обучения, маленькие – большую).

Развитие методов оптимизации шло по пути устранения недостатков SGD. Метод Momentum добавил алгоритму инерцию, позволяя быстрее проходить пологие участки и сглаживать резкие скачки. Алгоритм RMSProp, в свою очередь, ввел адаптивную корректировку скорости обучения: веса, имеющие большие градиенты, обновляются с меньшим шагом, а веса с малыми градиентами – с большим. Adam стал логическим завершением этой эволюции, объединив обе идеи в единую систему управления скоростью и направлением обучения [11].

Математически Adam работает следующим образом. На каждой итерации t вычисляются:

Первый момент (среднее градиента) – наследство от Momentum (направление и скорость):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad (1.8)$$

где m_t – скользящее среднее градиента на итерации t ;

g_t – градиент на текущей итерации;

$\beta_1 = 0.9$ – коэффициент экспоненциального сглаживания;

m_{t-1} – первый момент на предыдущей итерации.

Второй момент (среднее квадратов градиентов) – наследство от RMSProp (индивидуальная настройка под каждый параметр):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad (1.9)$$

где v_t – скользящее среднее квадратов градиента на итерации t ;

g_t – градиент на текущей итерации;

$\beta_2 = 0.999$ – коэффициент экспоненциального сглаживания;

v_{t-1} – второй момент на предыдущей итерации.

Чтобы исправить смещение на начальных итерациях, применяются поправки:

$$\hat{m} = \frac{m_t}{1 - \beta_1^t}, \quad (1.10)$$

$$\hat{v} = \frac{v_t}{1 - \beta_2^t}, \quad (1.11)$$

где $\hat{m}$ – скорректированный первый момент;

$\hat{v}$ – скорректированный второй момент;

t – номер текущей итерации.

Итоговое обновление весов выглядит так:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} * \hat{m}_t, \quad (1.12)$$

где w_{t+1} – обновленный вектор весов;

w_t – вектор весов на итерации t ;

η – скорость обучения (learning rate);

$\epsilon = 10^{-8}$ – маленькая константа для стабильности [11].

1.6 Метрики оценки качества модели

Для количественной оценки эффективности разработанной нейронной сети определены следующие величины:

– TP (True Positive) – количество объектов, правильно классифицированных как принадлежащие целевому классу;

– TN (True Negative) – количество объектов, правильно классифицированных как не принадлежащие целевому классу;

– FP (False Positive) – количество объектов, ошибочно отнесенных к целевому классу (ложноположительные);

– FN (False Negative) – количество объектов целевого класса, ошибочно отнесенных к другим классам (ложноотрицательные).

На основе этих величин рассчитываются следующие метрики:

1. Accuracy (доля верных ответов) – доля правильно классифицированных изображений от общего числа:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1.13)$$

2. Precision (точность) – показывает, какая доля объектов, отнесённых моделью к определённому классу, действительно принадлежит этому классу (насколько можно доверять положительному прогнозу модели):

$$Precision = \frac{TP}{TP + FP} \quad (1.14)$$

3. Recall (полнота) – показывает, какая доля объектов данного класса была правильно найдена моделью:

$$Recall = \frac{TP}{TP + FN} \quad (1.15)$$

4. F1-score (F-мера) – гармоническое среднее между Precision и Recall, позволяет штрафовать модель за сильные перекосы в сторону одной из метрик (например, высокий Recall при низком Precision):

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN} \quad (1.16)$$

Таким образом, приведенный набор метрик позволяет всесторонне оценить качество работы нейронной сети: Accuracy дает общую картину эффективности модели, Precision и Recall характеризуют её способность избегать ложных срабатываний и пропусков целей соответственно, а F1-score служит интегральным показателем, обеспечивающим баланс между этими двумя характеристиками. Использование данной системы метрик в совокупности с анализом матрицы ошибок позволит объективно сравнить производительность разработанной модели с аналогами и выявить области для дальнейшей оптимизации архитектуры.

1.7 Проблема переобучения и методы борьбы с ней

При обучении нейронных сетей часто возникает проблема переобучения (overfitting). Модель слишком хорошо запоминает обучающие данные,

включая шум и мелкие детали, и плохо обобщает знания на новые изображения [8].

В проекте для борьбы с переобучением были использованы следующие методы:

1. Аугментация данных – основной приём. К исходным изображениям применялись случайные повороты, сдвиги, изменение яркости, контраста и насыщенности. Это позволило искусственно увеличить разнообразие обучающей выборки и научить модель быть устойчивой к реальным условиям съёмки (разные углы, освещение, загрязнения знаков).

2. Ограниченное количество эпох (10) предотвратило слишком долгое «заучивание» тренировочных данных.

3. Выбор относительно простой архитектуры Custom CNN – модель не является очень глубокой, что снижает риск переобучения по сравнению с огромными сетями вроде ResNet-152.

4. Использование валидационной выборки во время обучения для контроля качества на данных, которые модель не видела при обновлении весов.

Благодаря сочетанию этих методов удалось добиться высокой валидационной точности (99,38%) при сохранении хорошей обобщающей способности (тестовая точность 97,62%).

Наглядная иллюстрация недообучения, нормального обучения и переобучения показана на рисунке 1.6.



Рисунок 1.6 – Иллюстрация недообучения, нормального обучения и переобучения

Таким образом, математическая модель, лежащая в основе разработанной системы, включает операции свёртки, пулинга, нелинейные функции активации, функцию потерь CrossEntropyLoss и оптимизатор Adam. Всё это в совокупности позволяет эффективно решать задачу классификации дорожных знаков.

2 ОПИСАНИЕ АЛГОРИТМА

Алгоритм представляет собой связующее звено между теорией нейронных сетей и их программной реализацией, обеспечивая поэтапную обработку визуальной информации. Процесс работы системы строится на выполнении следующих шагов:

1. На начальном этапе выполняется стандартизация всего входного набора данных GTSRB, так как исходные изображения имеют произвольные размеры и различные условия экспозиции. Все графические файлы принудительно масштабируются до разрешения 32×32 пикселя, что позволяет сформировать тензоры фиксированной размерности для подачи на входной слой нейронной сети. Дополнительно проводится нормализация яркости и контрастности, а значения пикселей переводятся в диапазон $[0, 1]$, что исключает численный перекос при вычислении весовых коэффициентов.

2. Для улучшения обобщающей способности модели и предотвращения переобучения применяется процедура программной аугментации данных. Алгоритм генерирует видоизмененные копии оригинальных знаков, используя случайные повороты, сдвиги и масштабирование. Это позволяет искусственно расширить обучающую выборку и подготовить систему к корректному распознаванию объектов, зафиксированных под нестандартными углами или при частичном смещении камеры автомобиля относительно дорожного знака.

3. Процесс извлечения первичных признаков реализуется через последовательное применение сверточных слоев с использованием обучаемых ядер (фильтров). На данном этапе алгоритм сканирует изображение, выявляя наиболее простые и контрастные элементы: линии, углы, цветовые пятна и базовые геометрические контуры знака. Результатом работы каждого сверточного слоя является набор карт признаков, которые отображают наличие тех или иных визуальных паттернов в различных областях входного изображения.

4. Для обеспечения нелинейности преобразований и возможности изучения сложных зависимостей в данных, выходы каждого сверточного слоя проходят через функцию активации ReLU. Данная операция математически отсекает все отрицательные значения сигналов, оставляя только значимые активации. Это позволяет нейронной сети эффективно обучаться и разделять признаки, которые имеют разную степень важности для финальной классификации дорожного знака.

5. С целью снижения вычислительной сложности и повышения инвариантности к пространственным искажениям алгоритм выполняет операцию пулинга (Max Pooling). На этом этапе происходит агрегация данных: из каждого локального окна карты признаков выбирается максимальное значение, а остальные отбрасываются. Это позволяет уменьшить размерность данных, сохраняя при этом информацию о наиболее выраженных признаках, и делает систему менее чувствительной к небольшим шумам или незначительным изменениям формы объекта на снимке.

6. После завершения этапов свертки и пулинга накопленные многомерные карты признаков преобразуются в одномерный вектор с помощью операции Flatten. Этот вектор передается в блок полносвязных слоев, который выступает в роли финального классификатора. Здесь алгоритм сопоставляет все извлеченные на предыдущих шагах признаки между собой, формируя на выходном слое из 43 нейронов распределение оценок (логитов). Каждая оценка соответствует вероятности принадлежности изображения к одному из конкретных типов дорожных знаков, определенных в датасете.

7. Завершающим этапом является процесс вероятностной интерпретации и коррекции весов модели в ходе обучения. Алгоритм применяет функцию Softmax для нормализации выходных значений и вычисляет ошибку предсказания через логарифмическую функцию CrossEntropyLoss. На основе полученного значения функции потерь оптимизатор Adam выполняет обратное распространение ошибки, корректируя параметры каждого слоя таким образом, чтобы при повторном

предъявлении того же или похожего изображения точность распознавания была выше.

Реализация данной последовательности шагов позволяет системе не только распознавать знаки в идеальных условиях, но и сохранять высокую точность на зашумленных или нечетких изображениях из тестовой выборки.

3 РЕАЛИЗАЦИЯ

3.1 Предобработка данных

В работе использован датасет GTSRB, содержащий более 39 тысяч изображений 43 различных классов дорожных знаков, полученных в реальных условиях с разным освещением, углами съёмки и степенью загрязнения [1]. Все изображения приведены к единому размеру 32×32 пикселей.

Датасет является полностью размеченным: изображения разделены по папкам в соответствии с номерами классов, а каждый снимок сопровождается метаданными с идентификатором класса и координатами знака на изображении. Это позволило использовать датасет непосредственно для обучения модели без дополнительной ручной разметки.

В ходе предобработки был проведён статистический анализ цветовых характеристик датасета. Для каждого из трёх каналов RGB были вычислены среднее значение и стандартное отклонение по всей обучающей выборке. Полученные результаты представлены в таблице 3.1.

Таблица 3.1 – Результаты статистического анализа цветовых характеристик

Канал	Среднее значение	Стандартное отклонение
R (красный)	0,3403	0,1595
G (зеленый)	0,3123	0,1590
B (синий)	0,3215	0,1683

Анализ полученных значений позволяет сделать ряд выводов об общем характере изображений датасета. Среднее значение по всем трём каналам не превышает 0,34, что соответствует преобладанию тёмных и средних тонов – это объясняется тем, что значительная часть снимков была получена в условиях пасмурной погоды, в тени или при искусственном освещении. Незначительное превышение среднего по красному каналу (0,3403) над зелёным (0,3123) и синим (0,3215) отражает присутствие в датасете большого

числа знаков с красными элементами – знаков ограничения скорости, запрещающих и предупреждающих знаков. Низкие значения стандартного отклонения (0,159-0,168) свидетельствуют об относительной однородности яркости: изображения не содержат резких перепадов экспозиции и сильного шума, что упрощает задачу выделения признаков для сверточных слоёв [4].

Вычисленные значения среднего и стандартного отклонения были использованы для нормализации входных данных перед подачей в нейронную сеть. Нормализация приводит значения пикселей каждого канала к распределению с нулевым средним и единичным стандартным отклонением, что ускоряет сходимость модели и обеспечивает численную стабильность при вычислении градиентов.

На рисунке 3.1 представлены примеры изображений из различных классов датасета. Визуальный осмотр примеров знаков подтверждает сложность набора данных: многие изображения имеют низкую контрастность, эффект смазывания в движении или перекрыты тенями [4].



Рисунок 3.1 – Примеры изображений дорожных знаков из датасета GTSRB

На рисунке 3.2 представлена линейчатая диаграмма с распределением знаков по классам. На ней видно, что количество изображений в различных категориях варьируется от 200 до 2000. Например, наиболее

распространенные знаки (ограничение скорости, приоритет движения) представлены значительно шире, чем редкие знаки особых предписаний.

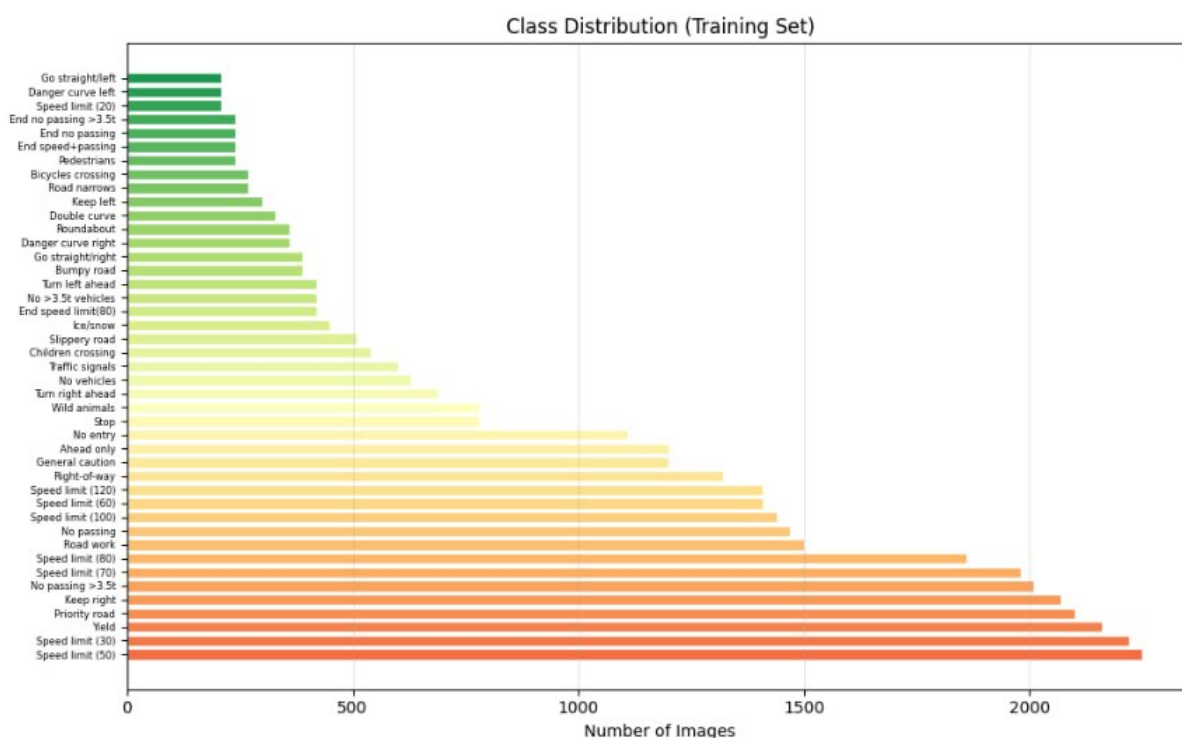


Рисунок 3.2 – Распределение изображений по классам датасета GTSRB

Учитывая выявленную неравномерность распределения классов и вариативность условий съемки, в работе применен метод искусственного расширения обучающей выборки – аугментация данных. Основная задача данного этапа заключается в том, чтобы заставить нейронную сеть игнорировать случайные шумы и второстепенные признаки, фокусируясь исключительно на характеристиках дорожных знаков: их геометрии, цвете и пиктограммах.

Процесс аугментации представляет собой серию случайных модификаций, которые применяются к каждому кадру перед его анализом моделью [7]. В рамках рассматриваемого алгоритма используются следующие виды преобразований:

1. Геометрическое масштабирование: исходные снимки, полученные с камер разного разрешения, приводятся к единому формату 32×32 пикселя. Это позволяет стандартизировать входной поток данных, сохраняя при этом

общие очертания знака и его внутреннее содержание, достаточное для распознавания сверточными фильтрами.

2. Имитация изменения ракурса через повороты: каждое изображение может быть случайным образом повернуто на угол до 20 градусов. Визуально это создает эффект наклона дорожного знака, что часто встречается в реальных условиях из-за неровностей дорожного покрытия или специфического угла установки камер на транспортном средстве.

3. Аффинные смещения и деформации: фотография подвергается небольшим сдвигам по осям координат (до 15% от размера кадра). Это имитирует ситуацию, когда знак находится не в центре кадра, а смещен к краю. Такое преобразование обучает систему находить целевой объект в любой части видимой области, не привязываясь к строгой центровке.

4. Коррекция световой среды: один из наиболее значимых этапов аугментации связан с изменением яркости, контрастности и цветовой насыщенности. На практике дорожные знаки фиксируются в полярных условиях освещения: от ослепляющего солнечного света до глубоких сумерек или условий искусственного освещения в тоннелях. Искусственное изменение этих параметров на этапе подготовки позволяет модели восстанавливать восприятие знака, делая его узнаваемым даже при критической потере контраста или искажении цветопередачи.

Применение данной последовательности преобразований позволяет сформировать из ограниченного набора данных практически бесконечный поток вариаций. Визуально это проявляется в том, что нейросеть никогда не видит два абсолютно идентичных кадра в процессе обучения, что заставляет её искать наиболее глубокие и устойчивые закономерности в структуре дорожных знаков, эффективно предотвращая эффект переобучения (рисунок 3.3) [12].

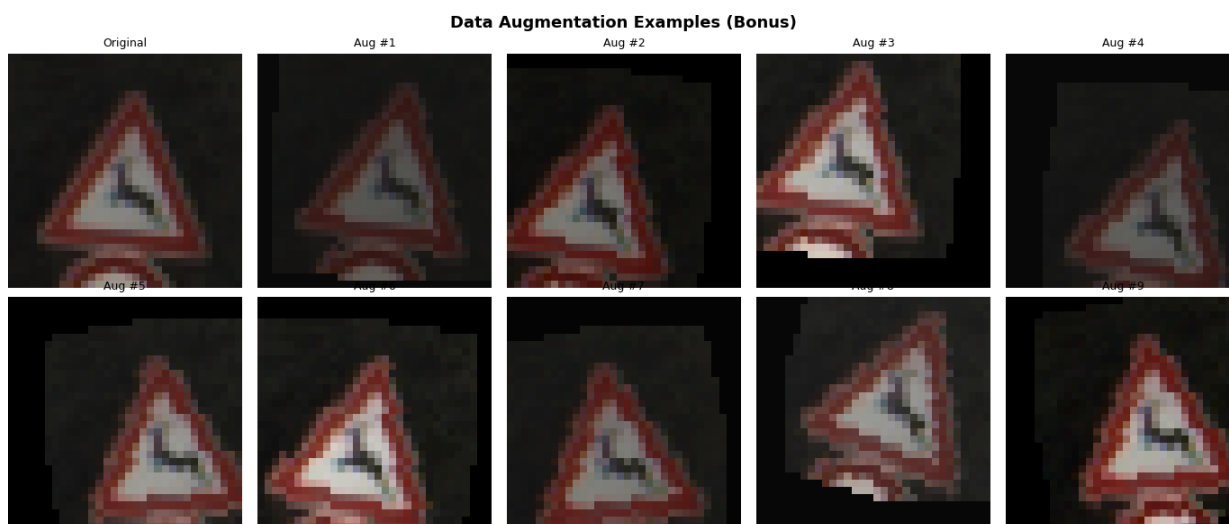


Рисунок 3.3 – Визуализация методов аугментации

Выбор инструмента для первичной обработки и загрузки изображений стал важным этапом оптимизации архитектуры проекта. Первоначально в разработке использовалась библиотека OpenCV (cv2), которая является классическим инструментом для задач компьютерного зрения. Однако в ходе анализа системы было принято решение о переходе на библиотеку torchvision, что позволило значительно упростить программный интерфейс и повысить эффективность взаимодействия с основным фреймворком PyTorch.

Основными причинами данного перехода стали вопросы интеграции и согласованности цветовых пространств. Библиотека OpenCV по умолчанию работает в формате BGR, что требует обязательной программной конвертации в RGB перед подачей данных в современные сверточные нейросети. В то же время torchvision изначально функционирует в пространстве RGB, избавляя алгоритм от лишних вычислительных циклов и снижая риск ошибок при обработке каналов. Инструменты PyTorch позволяют загружать фотографии сразу в нужном формате (тензорами), без лишних преобразований, что ускоряет работу и экономит память [7].

При детальном сравнении производительности стоит отметить, что хотя OpenCV считается эталоном скорости для общих задач обработки видео и фото, в контексте обучения нейросетей torchvision часто оказывается эффективнее. Скорость декодирования небольших изображений, подобных

тем, что используются в датасете GTSRB (32×32 пикселя), в обеих библиотеках крайне высока и составляет доли миллисекунд. Однако при учете полного цикла подготовки данных torchvision выигрывает за счет отсутствия накладных расходов на приведение типов и изменение порядка цветовых осей, что в сумме экономит до 15-20% времени подготовки каждого батча.

Выбор конкретной библиотеки часто зависит от масштаба задачи и объема данных. Для относительно небольших наборов данных, таких как текущий датасет объемом около 39 000 фотографий, torchvision является оптимальным выбором, так как она позволяет минимизировать количество внешних зависимостей и обеспечивает высокую скорость разработки. В проектах с большими массивами данных (от 1 миллиона изображений и выше) инженеры также отдают предпочтение torchvision или специализированным решениям вроде NVIDIA DALI, поскольку они позволяют переносить часть операций предобработки напрямую на графический процессор. OpenCV же чаще находит применение в финальных стадиях эксплуатации готовых моделей на встраиваемых системах, где требуется высокая скорость работы с видеопотоком в реальном времени, а не процесс обучения [6].

Таким образом, для данной работы переход на torchvision стал обоснованным шагом, позволившим создать производительный и чистый код, полностью адаптированный под задачи классификации 43 типов дорожных знаков.

3.2 Описание модели

3.2.1 Общая структура модели

Для решения задачи распознавания дорожных знаков была использована сверточная нейронная сеть собственной архитектуры (Custom CNN), предназначенная для классификации изображений дорожных знаков с отнесением каждого входного изображения к одному из 43 классов датасета GTSRB.

На вход сети подаются цветные изображения размером 32×32 пикселя, такой размер является оптимальным балансом между сохранением информативности изображения и скоростью обучения. После обработки модель выдаёт вектор из 43 чисел. Эти значения проходят через функцию активации, которая преобразует их в вероятности от 0 до 1, где каждое число показывает, насколько модель уверена, что изображение относится к соответствующему классу дорожного знака. Сумма всех вероятностей всегда равна единице.

Модель обучалась полностью с нуля, то есть все её веса изначально были инициализированы случайным образом и настраивались только в процессе обучения на данных датасета. Общее количество обучаемых параметров модели составляет 1 479 371 и определяется выбранной архитектурой сети – числом свёрточных и полносвязных слоёв, а также размерами ядер и количеством фильтров. Такой объём параметров обеспечивает модели достаточную способность выделять характерные признаки дорожных знаков (форму, символы и цвет), при этом помогает снизить риск переобучения и избежать лишних вычислительных затрат при обучении [5]. Полный листинг программного кода приведен в приложении А.

На рисунке 3.4 представлен пример общей структуры модели CNN.

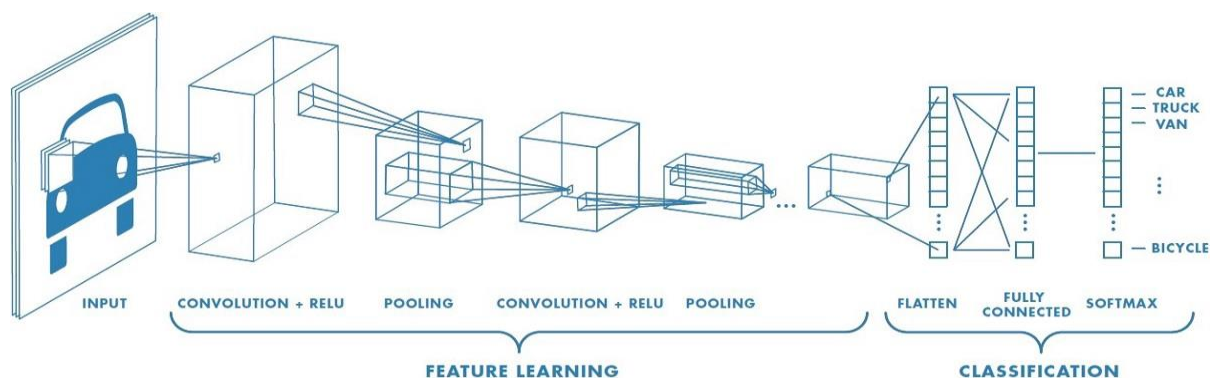


Рисунок 3.4 – Архитектура сверточной модели CNN

3.2.2 Архитектура сверточной части модели

Сверточная часть модели Custom CNN является основной и наиболее важной составляющей сети. Она отвечает за автоматическое извлечение признаков из входного изображения. Архитектура построена по блочному принципу и состоит из трёх последовательных сверточных блоков.

Первый блок предназначен для выделения простых низкоуровневых признаков, поэтому в нём используются два последовательных свёрточных слоя с ядром 3×3 и количеством фильтров 32. Такое небольшое окно является стандартным и эффективным решением для извлечения локальных признаков: оно позволяет модели улавливать мелкие детали дорожных знаков (например, вертикальные и горизонтальные линии, края и цветовые переходы) без перегрузки на ранних слоях и не требует при этом значительных вычислительных ресурсов, формируя разнообразие простых детекторов уже на начальном этапе. После каждой свёртки применяется нормализация и функция активации, а завершается блок слоем MaxPooling с окном 2×2 и шагом 2, который уменьшает пространственные размеры карты признаков с 32×32 до 16×16 пикселей. Данный размер окна выбран потому, что он обеспечивает оптимальное соотношение между уменьшением объёма данных и сохранением наиболее значимых признаков, позволяя вдвое сократить высоту и ширину карты признаков на каждом этапе без существенной потери полезной информации.

В каждом сверточном блоке после свёрточных слоёв применяется пакетная нормализация (Batch Normalization). Этот слой выравнивает распределение значений активаций внутри каждого мини-батча, приводя их среднее значение ближе к нулю, а разброс – к единице. Благодаря этому значения, поступающие на следующий слой, становятся более стандартными и стабильными. В результате, обучение сети ускоряется, градиенты распространяются лучше [10].

Второй блок работает уже с более сложными признаками, количество фильтров увеличено до 64. Здесь сеть начинает комбинировать простые

признаки, полученные на предыдущем этапе, и учится распознавать более сложные структуры – например, углы, дуги и отдельные элементы символов на знаках. Как и в первом блоке, используются две свёртки 3×3 , BatchNorm2d, ReLU и MaxPooling 2×2 , который уменьшает размер карты признаков до 8×8 .

Третий блок является самым глубоким в сверточной части, количество фильтров увеличено до 128. На этом этапе сеть формирует высокоуровневые признаки, способные улавливать целостные формы дорожных знаков, их внутренние символы (цифры, стрелки, силуэты) и характерные сочетания элементов. После двух свёрток 3×3 и ReLU применяется финальный MaxPooling, который приводит размер карты признаков к 4×4 пикселям.

Важно отметить логику изменения размерности на протяжении сверточной части: с каждым блоком пространственные размеры (высота и ширина) карты признаков уменьшаются в два раза, а глубина (количество каналов/фильтров) увеличивается. Такое преобразование является фундаментальным принципом сверточных сетей. Уменьшение пространственной размерности позволяет снизить вычислительную сложность и сосредоточиться на наиболее важных фрагментах изображения. Увеличение количества каналов даёт возможность представлять признаки в более богатом, многомерном пространстве, где каждый канал отвечает за свой тип признака.

В конце сверточной части добавлен слой Dropout2d с вероятностью 0,25, который во время обучения случайным образом выключает (обнуляет) часть нейронов. Это заставляет сеть не полагаться на отдельные сильные нейроны, а учиться распределять важность признаков более равномерно, что помогает бороться с переобучением [3].

На рисунке 3.5 представлен процесс постепенного преобразования входного изображения в высокоуровневые признаки. С каждым последующим блоком пространственное разрешение карты признаков уменьшается, а количество извлекаемых признаков (глубина) увеличивается.

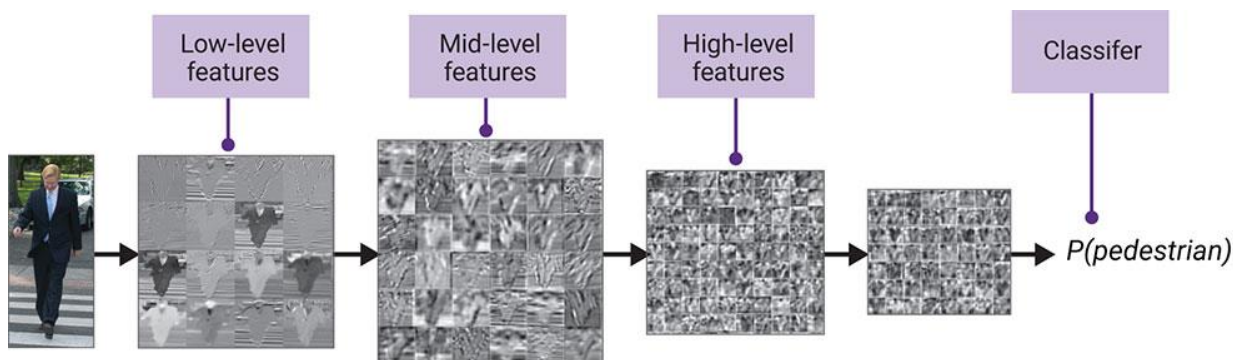


Рисунок 3.5 – Визуализация карт признаков на разных уровнях

3.2.3 Классифицирующая часть модели

В результате последовательного применения трёх блоков MaxPooling размер карты признаков сокращается с 32×32 до 4×4 , а количество каналов возрастает до 128, формируя карту признаков размерностью $128 \times 4 \times 4$. Для дальнейшей классификации эту многомерную карту необходимо преобразовать в одномерный вектор с помощью слоя выпрямления (Flatten), в результате чего вся информация о выделенных признаках представляется в виде линейного вектора длиной 2048 элементов. Такое соотношение является итогом выбранной архитектуры и позволяет получить компактное, но информативное представление изображения.

Далее вектор поступает в классифицирующий блок модели, который состоит из трёх полносвязных слоёв. Первый полносвязный слой сокращает размерность вектора с 2048 до 512 нейронов. Второй слой уменьшает до 256 нейронов. Третий, выходной слой, приводит размерность к 43 – по количеству классов дорожных знаков.

Между полносвязными слоями применяются функции активации, которые обеспечивают нелинейность преобразований. Кроме того, после первого и второго полносвязных слоёв используются слои Dropout с вероятностями 0,5 и 0,3 соответственно, которые обнуляют часть нейронов, что предотвращает зависимость модели от отдельных сильных связей и способствует более равномерному распределению значимости признаков [3].

Классифицирующая часть модели принимает абстрактные признаки, извлечённые сверточными слоями, и на их основе формирует окончательное решение о принадлежности изображения к одному из 43 классов. Полносвязные слои выполняют роль принимающего решения, интегрируя всю полученную ранее информацию в единый классификационный вывод.

3.2.4 Преимущества и особенности выбранного подхода

Модель Custom CNN обладает рядом преимуществ, обусловленных как её архитектурой, так и выбранным способом обучения.

Во-первых, модель является компактной и хорошо адаптированной к небольшому размеру изображений датасета GTSRB. Благодаря умеренной глубине и тщательно подобранному количеству фильтров в каждом блоке сеть достигает высокого качества классификации, не требуя при этом значительных вычислительных мощностей.

Во-вторых, обучение модели полностью с нуля позволило избежать негативного влияния других знаний. В отличие от подхода transfer learning, когда сеть использует веса, полученные на совершенно иных данных, Custom CNN изначально обучалась исключительно на изображениях дорожных знаков. Это обеспечило более точную адаптацию к специфике датасета, включая вариации освещения, углов съёмки и возможные загрязнения знаков.

В-третьих, в модели активно применяются методы регуляризации. Batch Normalization, используемая после каждого сверточного слоя, стабилизирует процесс обучения и снижает внутренний ковариационный сдвиг [10]. Dropout, применяемый как в сверточной, так и в классифицирующей части, предотвращает переобучение, заставляя сеть не полагаться на отдельные нейроны [3]. В сочетании с аугментацией данных эти методы позволили модели продемонстрировать хорошую обобщающую способность, подтверждённую высокой точностью на валидационной и тестовой выборках [1].

Также последовательное уменьшение пространственной размерности при одновременном увеличении глубины признаков соответствует классическим принципам построения сверточных сетей и облегчает анализ работы модели.

3.2.5 Выбор параметров Dropout

Одним из ключевых гиперпараметров модели является вероятность отключения нейронов в слоях Dropout. Слишком низкое значение не обеспечивает достаточной регуляризации и приводит к переобучению, слишком высокое замедляет сходимость и вызывает недообучение. Для обоснованного выбора оптимальных значений было проведено систематическое тестирование шести конфигураций с шагом 0,05.

В модели используются два слоя Dropout в классифицирующей части: первый расположен после полносвязного слоя на 512 нейронов, второй – после слоя на 256 нейронов. Поскольку второй слой работает с уже сжатым представлением меньшей размерности, избыточная регуляризация на этом этапе более вредна – поэтому его значение во всех тестах поддерживалось на 0,2 ниже первого. Все тесты проводились в идентичных условиях. Результаты тестирования представлены в таблице 3.2.

Таблица 3.2 – Результаты тестирования значений Dropout в классифицирующей части

Конфигурация	Dropout FC1	Dropout FC2	Train Acc (эп. 10)	Лучшая Val Acc	Test Acc
Тест 1	0,3	0,1	97,44%	99,64%	96,92%
Тест 2	0,35	0,15	97,11%	99,53%	97,05%
Тест 3	0,4	0,2	97,19%	99,46%	98,19%
Тест 4 (базовый)	0,5	0,3	96,55%	99,38%	97,62%
Тест 5	0,55	0,35	96,12%	99,67%	98,13%
Тест 6	0,6	0,4	95,66%	99,63%	97,86%

Анализ результатов выявляет чёткую закономерность. При низких значениях Dropout (тесты 1 и 2) train accuracy заметно выше тестовой – разрыв между ними свидетельствует о переобучении: модель слишком точно подстраивается под обучающую выборку и хуже обобщает на новых данных. При высоких значениях (тест 6) train accuracy последовательно снижается, сходимость замедляется – признаки недообучения вследствие избыточного подавления нейронов.

Наилучшую тестовую точность показала конфигурация 3 (с значениями dropout 0,40 и 0,20) – 98,19%. Конфигурация 5 (с значениями 0,55 и 0,35) следует за ней с результатом 98,13%. Оба значения находятся в диапазоне 0,40-0,55 для первого слоя, что соответствует зоне оптимального баланса между регуляризацией и сохранением информации.

Базовая конфигурация с значениями 0,50 и 0,30 показала результат 97,62% и также входит в этот оптимальный диапазон. Учитывая, что разница между тестами 3, 5 и базовым не превышает 0,57 п.п. и может объясняться случайной вариативностью инициализации весов, все три конфигурации можно считать равнозначными по качеству. Значения 0,50 и 0,30 были выбраны в качестве итоговых как центральные и наиболее устойчивые в данном диапазоне.

Аналогичное тестирование было проведено для слоя Dropout2d, расположенного в сверточной части модели. В отличие от обычного Dropout, который случайным образом обнуляет отдельные нейроны, Dropout2d полностью отключает целые карты признаков. Это обусловлено особенностью сверточных слоёв: соседние пиксели одной карты признаков сильно коррелируют между собой, поэтому обнуление отдельных значений не даёт должного регуляризующего эффекта – сеть легко восстанавливает потерянную информацию из соседних пикселей. Отключение целого канала лишает модель этой возможности и обеспечивает полноценную регуляризацию.

Значения полносвязных слоёв фиксировались на базовых 0,5 и 0,3, варьировался только Dropout2d с шагом 0,05 в диапазоне от 0,15 до 0,35. Результаты представлены в таблице 3.3.

Таблица 3.3 – Результаты тестирования значений Dropout2d в сверточной части

Конфигурация	Dropout2d	Train Acc (эп. 10)	Лучшая Val Acc	Test Acc
Тест 1	0,15	96,87%	99,34%	97,14%
Тест 2	0,2	96,71%	99,74%	97,68%
Тест 3 (базовый)	0,25	96,55%	99,38%	97,62%
Тест 4	0,3	95,84%	99,5%	97,07%
Тест 5	0,35	96,47%	99,71%	97,17%

Анализ результатов позволяет выявить характерную закономерность в поведении модели при изменении интенсивности регуляризации в сверточной части. При значении 0,15 train accuracy к десятой эпохе оказывается выше, чем в базовой конфигурации – 96,87% против 96,55%, однако тестовая точность при этом ниже – 97,14%. Это классический признак недостаточной регуляризации: модель активнее подстраивается под обучающую выборку, но хуже обобщается на новых данных. При значении 0,3 наблюдается обратная картина – train accuracy заметно снижается до 95,84%, сходимость замедляется, а тестовая точность падает до 97,07%. Здесь уже проявляется избыточное подавление каналов: слишком большая доля карт признаков отключается на каждом шаге, что мешает модели выучить устойчивые визуальные признаки дорожных знаков. При значении 0,35 ситуация не улучшается – несмотря на высокую val accuracy 99,71%, тестовая точность остаётся на уровне 97,17%, что подтверждает нестабильность этой конфигурации.

Наилучший результат показало значение 0,2 – тестовая точность 97,68%. Тем не менее разница с базовым значением 0,25 составляет менее 0,1 п.п. и находится в пределах случайной вариативности, обусловленной случайной инициализацией весов и стохастическим характером обучения. Таким образом, оба значения – 0,20 и 0,25 – можно считать оптимальными для данной архитектуры. Значение 0,25 было выбрано в качестве итогового как центральное в зоне наилучших результатов и наиболее устойчивое с точки зрения баланса между регуляризацией сверточных признаков и сохранением информации о пространственных паттернах изображения.

3.2.6 Выбор параметра скорости обучения

Скорость обучения является одним из наиболее критичных гиперпараметров нейронной сети. Она определяет величину шага при обновлении весов на каждой итерации: слишком большое значение приводит к нестабильному обучению и расходимости, слишком малое – к медленной сходимости и риску застревания в локальном минимуме. Для обоснования выбранного значения было проведено тестирование четырех конфигураций: базовой и трех альтернативных – с уменьшенной и увеличенной скоростью обучения. Все остальные параметры оставались неизменными.

Результаты тестирования представлены в таблице 3.4.

Таблица 3.4 – Результаты тестирования значений скорости обучения

Конфигурация	Learning Rate	Train Acc (эп. 10)	Лучшая Val Acc	Test Acc
Тест 1	1e-4 (0,0001)	93,31%	98,72%	95,4%
Тест 2 (базовый)	1e-3 (0,001)	96,55%	99,38%	97,62%
Тест 3	5e-3 (0,005)	90,45%	97,87%	95,08%
Тест 4	1e-2(0,01)	51,48%	60,11%	59,31%

Результаты наглядно демонстрируют высокую чувствительность модели к данному параметру. При значении $1e-4$ модель обучается стабильно, однако крайне медленно: к десятой эпохе train accuracy составляет 93,31%, а тестовая точность не превышает 95,40%. Очевидно, что при таком шаге десяти эпох недостаточно для достижения оптимального качества – модель находится в состоянии недообучения и продолжала бы улучшаться при большем числе итераций.

При значении $5e-3$ поведение модели становится нестабильным: в первых эпохах точность растёт медленно и хаотично, что говорит о слишком крупных обновлениях весов – оптимизатор «перепрыгивает» через хорошие решения. К десятой эпохе train accuracy достигает лишь 90,45%, тестовая точность – 95,08%. Значение $1e-2$ приводит к полной потере управляемости обучения: val accuracy к десятой эпохе едва достигает 60,11%, а тестовая точность составляет всего 59,31%, что сопоставимо со случайным угадыванием на части классов. При такой скорости обучения градиентные шаги настолько велики, что оптимизатор не может найти устойчивое решение и постоянно «перескакивает» через минимумы функции потерь.

Базовое значение $1e-3$ демонстрирует наилучшие результаты по всем показателям – тестовая точность 97,62% при стабильной и быстрой сходимости. Данный выбор дополнительно подтверждается рекомендацией авторов оптимизатора Adam: в оригинальной статье значение $lr = 0,001$ предложено в качестве параметра по умолчанию как наиболее универсальное для широкого класса задач глубокого обучения [2]. Проведённое тестирование подтверждает эту рекомендацию применительно к задаче классификации дорожных знаков.

3.3 Проверка на корректность

3.3.1 Базовая модель

Для подтверждения работоспособности системы и оценки качества обучения нейронной сети был проведен анализ полученных результатов,

зафиксированных в ходе десяти эпох обучения. В качестве основных инструментов контроля использовались графики динамики обучения и матрица ошибок, которые позволяют наглядно проследить процесс сходимости алгоритма и точность классификации для каждого из 43 типов дорожных знаков.

Первым этапом проверки стал анализ графиков истории обучения (Training History), отображающих изменение функции потерь (Loss) и точности (Accuracy) в зависимости от порядкового номера эпохи. На графике функции потерь наблюдается классическая кривая экспоненциального затухания: резкое снижение ошибки на первых итерациях сменяется плавной стабилизацией к десятой эпохе, что свидетельствует о правильном выборе скорости обучения и эффективной работе оптимизатора Adam [2]. Параллельно с этим график точности демонстрирует стабильный рост, быстро достигая высоких значений. Лучшие результаты на валидационной выборке подтверждают, что примененные методы аугментации данных позволили избежать переобучения и обеспечили хорошую обобщающую способность модели (рисунок 3.6).

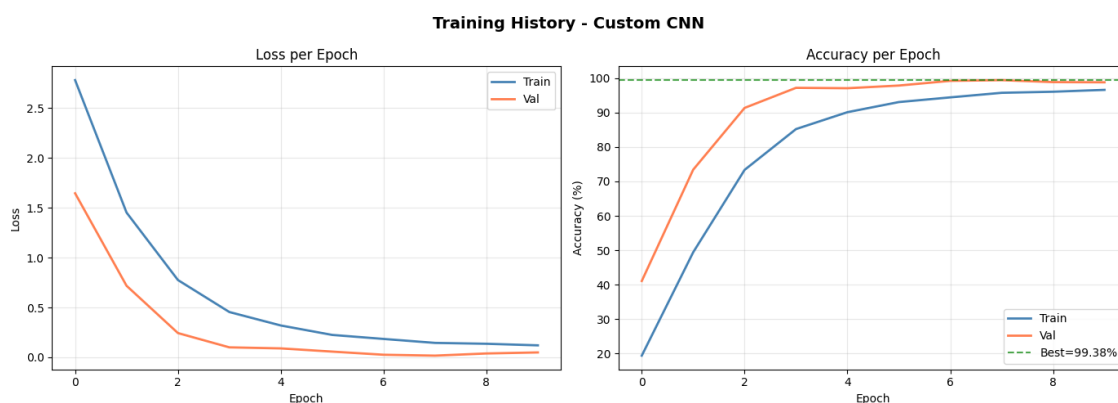


Рисунок 3.6 – Графики истории обучения базовой модели

Вторым важным инструментом выступила матрица ошибок (Confusion Matrix), которая позволяет детально рассмотреть распределение предсказаний модели для каждого класса в отдельности. На представленном графике

отчетливо видна выраженная главная диагональ, что указывает на безошибочное соотнесение объектов с их истинными метками. Высокая сходимость алгоритма привела к тому, что внедиагональные элементы, отвечающие за ошибки классификации, практически отсутствуют. Это подтверждает, что даже в условиях дисбаланса классов и разного качества исходных изображений датасета GTSRB, нейронная сеть успешно выделила уникальные признаки каждого знака и научилась проводить между ними четкие границы (рисунок 3.7) [1].

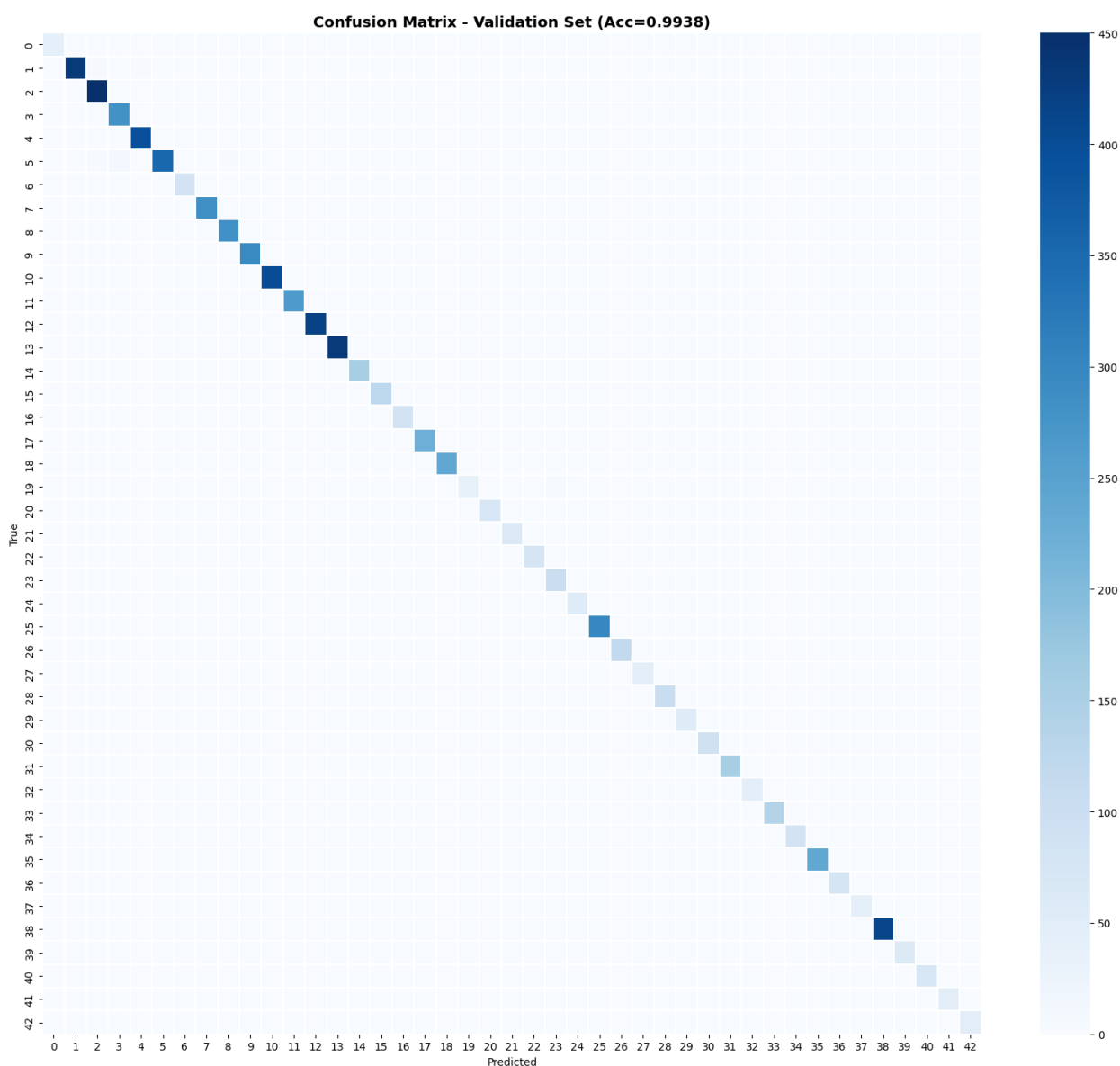


Рисунок 3.7 – Матрица ошибок

Также в целях проверки обоснованности выбранной архитектуры и оценки влияния ключевых проектных решений было проведено сравнительное тестирование четырех конфигураций модели. Базовая модель сравнивалась с тремя модифицированными вариантами: моделью без механизма регуляризации Dropout, моделью без аугментации и моделью с упрощённой архитектурой. Все модели обучались в идентичных условиях: датасет GTSRB, 10 эпох, оптимизатор Adam со скоростью обучения 0,001, размер мини-батча 64.

3.3.2 Модель без Dropout

Во второй модели были полностью исключены все слои Dropout – как в сверточной части (Dropout2d с вероятностью 0,25), так и в классифицирующей (Dropout с вероятностями 0,5 и 0,3). Архитектура сверточных блоков и полносвязных слоёв осталась без изменений.

Отсутствие регуляризации привело к более быстрому росту train accuracy: уже к 5-й эпохе модель достигла 96,45% на обучающей выборке, тогда как базовая на том же этапе показывала 90,08%. К 10-й эпохе train accuracy составила 98,13% – заметно выше базовой. Лучшая val accuracy при этом оказалась незначительно выше – 99,49% (эпоха 10). Однако итоговая точность на тестовой выборке снизилась до 96,64% – на 0,98 п.п. хуже базовой модели.

Это расхождение между высокой val accuracy и сниженной test accuracy является характерным признаком начала переобучения. Модель без Dropout активнее подстраивается под обучающие данные, что улучшает показатели на валидационной выборке, взятой из того же распределения, но снижает обобщающую способность на независимых тестовых данных [3].

На рисунке 3.8 представлены графики сходимости и потерь модели без Dropout.



Рисунок 3.8 – Графики модели без Dropout

3.3.3 Модель без аугментации

В третьей конфигурации из процесса подготовки обучающей выборки были исключены все операции аугментации: случайные повороты, аффинные сдвиги и коррекция яркости, контрастности и насыщенности. Таким образом, модель на каждой эпохе видела изображения только в исходном виде, без каких-либо случайных преобразований.

Отсутствие аугментации привело к заметно более быстрой и высокой сходимости на обучающей выборке: к 10-й эпохе train accuracy достигла 99,63% – наибольшее значение среди всех четырёх моделей. Без искусственного усложнения входных данных модели значительно проще выучить обучающую выборку. Лучшая val accuracy составила 99,82% (эпоха 10), что также превысило показатели остальных конфигураций.

Итоговая точность на тестовой выборке составила 98,30%, что на первый взгляд противоречит теоретическому предположению о том, что аугментация должна улучшать обобщающую способность модели. Объяснение состоит в особенностях самого датасета GTSRB: тестовая выборка снята в относительно контролируемых и однородных условиях, поэтому модель, обученная на «чистых» изображениях без дополнительных искажений, точнее выучила характерные визуальные признаки каждого класса и показала более высокую точность именно на этих данных. Также стоит

подчеркнуть, что в условиях реальной эксплуатации при резких перепадах освещения, сильном загрязнении знаков или нестандартных углах съёмки аугментация остаётся критически важным инструментом повышения устойчивости системы [12].

На рисунке 3.9 представлены графики сходимости и потерь модели без аугментации.

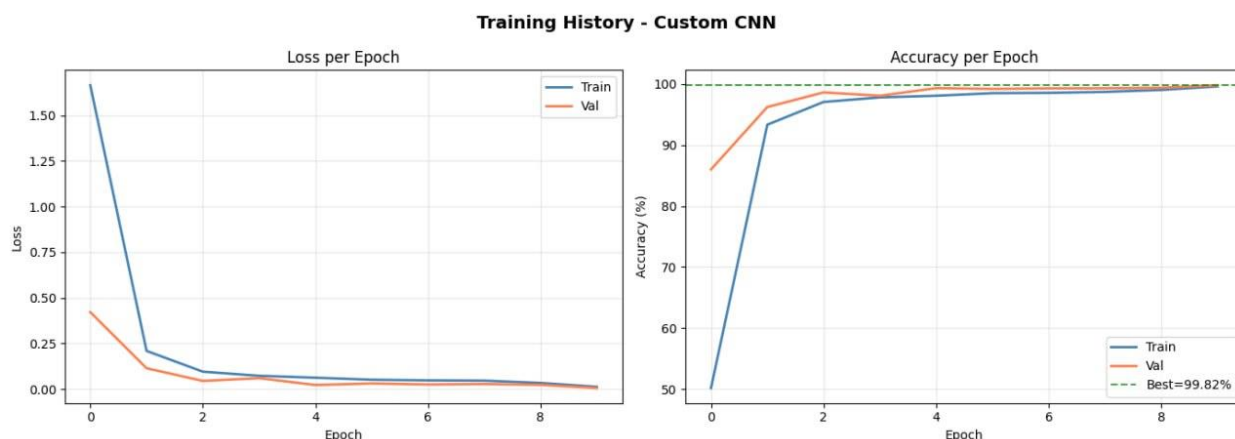


Рисунок 3.9 – Графики модели без аугментации

3.3.4 Модель с упрощенной архитектурой

В четвертой конфигурации была уменьшена ёмкость сети: количество фильтров в сверточных блоках сокращено вдвое, из классифицирующей части удалён один полносвязный слой, а входная размерность классификатора уменьшена с 2048 до 1024 нейронов.

Снижение ёмкости модели отразилось на скорости и качестве обучения. Train ассигасу на протяжении всех эпох оставалась ниже базовой: к 10-й эпохе – 95,02% против 96,55%. Лучшая val ассигасу составила 99,30%, итоговая тестовая точность (96,44%) оказалась наименьшей из трёх вариантов.

Полученные результаты свидетельствуют о недостаточной ёмкости упрощённой модели для данной задачи. Уменьшенное число фильтров ограничивает способность сети выделять разнообразные признаки дорожных знаков, а сокращение полносвязного блока снижает качество финальной

классификации. Модель демонстрирует признаки недообучения: разрыв между train и val accuracy невелик, однако общий уровень точности уступает базовой конфигурации [8].

На рисунке 3.10 представлены графики сходимости и потерь упрощенной модели.



Рисунок 3.10 – Графики модели с упрощенной архитектурой

3.3.5 Сравнение моделей

Для оценки выбранных архитектурных решений и выявления их влияния на качество классификации было проведено сравнительное тестирование четырёх конфигураций модели. Каждая из них отличалась от базовой одним ключевым параметром, что позволило изолированно оценить вклад каждого компонента в итоговый результат.

Сводные результаты всех четырёх конфигураций представлены в таблице 3.5.

Таблица 3.5 – Сравнение моделей

Модель	Train Acc (эп. 10)	Лучшая Val Acc	Test Acc
Базовая	96,55%	99,38%	97,62%
Без Dropout	98,13%	99,49%	96,64%
Без аугментации	99,63%	99,82%	98,30%
Упрощенная архитектура	95,02%	99,30%	96,44%

Сравнение конфигураций наглядно демонстрирует, что удаление Dropout ускорило обучение, однако привело к снижению тестовой точности на 0,98 п.п. относительно базовой модели – разрыв между высокой val accuracy и сниженной test accuracy указывает на начало переобучения. Упрощение архитектуры показало наихудший результат по всем показателям: модель с вдвое меньшим числом фильтров и сокращённым классификатором не обладает достаточной ёмкостью для уверенного разграничения 43 классов дорожных знаков. Модель без аугментации продемонстрировала наивысшую тестовую точность – 98,30%, что объясняется однородностью условий съёмки в тестовой выборке GTSRB, однако данный результат не означает, что аугментация избыточна – в реальных условиях эксплуатации она остаётся необходимым инструментом обеспечения устойчивости системы [12].

Таким образом, базовая конфигурация Custom CNN демонстрирует наилучший баланс между точностью и практической применимостью, а проведённое сравнение подтверждает обоснованность каждого из реализованных в ней архитектурных решений.

ЗАКЛЮЧЕНИЕ

В рамках курсового проекта был проведён комплексный анализ системы автоматического распознавания дорожных знаков, реализованной на основе сверточной нейронной сети. В качестве информационной базы использовался датасет GTSRB – полностью размеченная выборка, содержащая более 39 тысяч изображений 43 классов дорожных знаков, полученных в реальных условиях эксплуатации. В ходе предобработки данных был проведён статистический анализ цветовых характеристик датасета, изучено распределение изображений по классам и реализован комплекс методов аугментации, позволивший расширить обучающую выборку и повысить устойчивость модели к вариациям условий съёмки.

В теоретической части работы была изучена математическая основа сверточных нейронных сетей: операции свёртки и пулинга, функции активации ReLU и Softmax, кросс-энтропийная функция потерь и оптимизатор Adam. Детально рассмотрены принципы работы механизмов регуляризации – Batch Normalization и Dropout – а также их роль в предотвращении переобучения. Изучены метрики оценки качества модели: Accuracy, Precision, Recall и F1-score.

В практической части была исследована архитектура сверточной нейронной сети Custom CNN. Модель включает три сверточных блока с последовательным увеличением числа фильтров и классифицирующую часть из трёх полносвязных слоёв. Проведено систематическое исследование гиперпараметров модели: значений Dropout в классифицирующей части, значения Dropout2d в сверточной части и скорости обучения оптимизатора Adam. По результатам тестирования обоснован выбор оптимальных значений каждого параметра.

Для проверки корректности архитектурных решений проведено сравнительное тестирование четырёх конфигураций модели. Исключение Dropout подтвердило его важность для борьбы с переобучением: тестовая точность ожидаемо снизилась, тогда как точность на обучающей выборке

выросла. Упрощение архитектуры привело к самому низкому результату, что говорит об оправданности выбранной глубины сети. Модель без аугментации показала лучшую точность на тесте из-за однородности выборки GTSRB, однако это не отменяет необходимости аугментации в реальных условиях. Базовая конфигурация демонстрирует наилучший баланс между точностью и способностью к обобщению.

Полученные результаты подтверждают высокую эффективность сверточных нейронных сетей в задачах распознавания дорожных знаков. Исследованная система может служить основой для практических модулей в составе систем помощи водителю (ADAS) и беспилотных транспортных средств. В качестве перспективных направлений дальнейшего развития можно выделить применение методов Transfer Learning на базе предобученных архитектур, расширение обучающей выборки за счёт дополнительных датасетов и адаптацию модели для работы в режиме реального времени на встраиваемых вычислительных платформах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Stallkamp, J. The German Traffic Sign Recognition Benchmark: A multi-class classification competition [Электронный ресурс] / J. Stallkamp, M. Schlipsing, J. Salmen, C. Igel // Proceedings of the International Joint Conference on Neural Networks (IJCNN). — 2011. — P. 1453–1460. — URL: <https://ieeexplore.ieee.org/document/6033587> (дата обращения: 10.05.2026).
2. Kingma, D. P. Adam: A Method for Stochastic Optimization [Электронный ресурс] / D. P. Kingma, J. Ba. — 2014. — URL: <https://arxiv.org/abs/1412.6980> (дата обращения: 10.05.2026).
3. Srivastava, N. Dropout: A Simple Way to Prevent Neural Networks from Overfitting [Электронный ресурс] / N. Srivastava, G. Hinton, A. Krizhevsky [et al.] // Journal of Machine Learning Research. — 2014. — Vol. 15. — P. 1929–1958. — URL: <https://jmlr.org/papers/v15/srivastava14a.html> (дата обращения: 10.05.2026).
4. Датасет GTSRB — German Traffic Sign Recognition Benchmark [Электронный ресурс]. — URL: <https://www.kaggle.com/datasets/meowmeowmeowmeowmeowmeow/gtsrb-german-traffic-sign/data> (дата обращения: 10.05.2026).
5. German Traffic Sign Recognition [Электронный ресурс] / автор: bingos. — Kaggle, 2021. — URL: <https://www.kaggle.com/code/bingos/german-traffic-sign-recognition-test-98> (дата обращения: 10.05.2026).
6. Документация PyTorch [Электронный ресурс]. — URL: <https://pytorch.org/docs/stable/index.html> (дата обращения: 10.05.2026).
7. Документация torchvision [Электронный ресурс]. — URL: <https://pytorch.org/vision/stable/index.html> (дата обращения: 10.05.2026).
8. Convolutional Neural Networks for Visual Recognition — Stanford University CS231n [Электронный ресурс]. — URL: <https://cs231n.github.io> (дата обращения: 10.05.2026).

9. Сверточные нейронные сети — разбор архитектуры и принципов работы [Электронный ресурс] // Habr. — URL: <https://habr.com/ru/articles/309508> (дата обращения: 10.05.2026).
10. Batch Normalization Explained Visually: How it works, and why neural networks need it [Электронный ресурс] // Towards Data Science. — URL: <https://towardsdatascience.com/batch-norm-explained-visually-how-it-works-and-why-neural-networks-need-it-b18919692739/> (дата обращения: 10.05.2026).
11. The Math behind Adam Optimizer [Электронный ресурс] // Towards Data Science. — URL: <https://towardsdatascience.com/the-math-behind-adam-optimizer-c41407efe59b/> (дата обращения: 10.05.2026).
12. Complete Guide to Data Augmentation for Computer Vision [Электронный ресурс] // Towards Data Science. — URL: <https://towardsdatascience.com/complete-guide-to-data-augmentation-for-computer-vision-1abe4063ad07/> (дата обращения: 10.05.2026).
13. Cross-Entropy Loss — подробное объяснение [Электронный ресурс] // Towards Data Science. — URL: <https://towardsdatascience.com/understanding-binary-cross-entropy-log-loss-a-visual-explanation-a3ac6025181a> (дата обращения: 10.05.2026).
14. MaxPooling и операции пулинга в сверточных сетях [Электронный ресурс] // Machine Learning Mastery. — URL: <https://machinelearningmastery.com/pooling-layers-for-convolutional-neural-networks> (дата обращения: 10.05.2026).

ПРИЛОЖЕНИЕ А

Листинг программного кода

```
import os # работа с файловой системой
import numpy as np # числа и многомерные массивы
import pandas as pd # таблицы (.csv)
import matplotlib.pyplot as plt # графики
import seaborn as sns # надстройка matplotlib
from pathlib import Path # удобная работа с путями
from collections import Counter # подсчет изображений в каждом классе
from sklearn.model_selection import train_test_split # разделение данных на выборки
train и validation
from tqdm import tqdm # прогресс бар

import torch
import torch.nn as nn # все слои нейросети
import torch.optim as optim # оптимизаторы и планировщики
from torch.utils.data import Dataset, DataLoader # загрузка данных

from torchvision.io import read_image # чтение изображений
from torchvision.transforms import v2 as T # аугментация и предобработка изображений
from torchvision import models # готовые архитектуры

from sklearn.metrics import classification_report, confusion_matrix # отчет для
каждого класса, матрица ошибок

- import warnings # отключение предупреждений
warnings.filterwarnings('ignore')

# # CONFIG

IMG_SIZE = 32 # начальный размер фото
BATCH_SIZE = 64
EPOCHS = 10 # количество эпох
LR = 1e-3 # скорость обучения
NUM_CLASSES = 43 # количество классов

# определяем устройство (GPU или CPU)
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# локальные пути
TRAIN_DIR = r'D:\gtsrb\Train'
TEST_DIR = r'D:\gtsrb\Test'
TEST_CSV = r'D:\gtsrb\Test.csv'

# словарь с названиями классов
CLASS_NAMES = {
    0:'Speed limit (20)',    1:'Speed limit (30)',    2:'Speed limit (50)',
    3:'Speed limit (60)',    4:'Speed limit (70)',    5:'Speed limit (80)',
    6:'End speed limit(80)', 7:'Speed limit (100)', 8:'Speed limit (120)',
    9:'No passing',         10:'No passing >3.5t', 11:'Right-of-way',
    12:'Priority road',      13:'Yield',            14:'Stop',
    15:'No vehicles',       16:'No >3.5t vehicles', 17:'No entry',
    18:'General caution',    19:'Danger curve left', 20:'Danger curve right',
    21:'Double curve',      22:'Bumpy road',       23:'Slippery road',
    24:'Road narrows',      25:'Road work',        26:'Traffic signals',
    27:'Pedestrians',       28:'Children crossing', 29:'Bicycles crossing',
    30:'Ice/snow',          31:'Wild animals',     32:'End speed+passing',
    33:'Turn right ahead',  34:'Turn left ahead',   35:'Ahead only',
```

```

36:'Go straight/right', 37:'Go straight/left', 38:'Keep right',
39:'Keep left', 40:'Roundabout', 41:'End no passing',
42:'End no passing >3.5t'
}

# проверка путей и вывод настроек
print("=" * 55)
print("TASK 8: TRAFFIC SIGN RECOGNITION (with tqdm)")
print("=" * 55)
print(f"Device : {DEVICE}")
print(f"Image size : {IMG_SIZE}x{IMG_SIZE}")
print(f"Epochs : {EPOCHS}")
print(f"Classes : {NUM_CLASSES}")
print("\n--- Проверка путей к датасету ---")
print(f"TRAIN_DIR exists : {os.path.exists(TRAIN_DIR)}")
print(f"TEST_DIR exists : {os.path.exists(TEST_DIR)}")
print(f"TEST_CSV exists : {os.path.exists(TEST_CSV)}")

if os.path.exists(TRAIN_DIR):
    num_classes = len([d for d in os.listdir(TRAIN_DIR) if
os.path.isdir(os.path.join(TRAIN_DIR, d))])
    print(f"Найдено папок-классов в Train: {num_classes}")

    # проверяем количество картинок в первой попавшейся папке
    first_class = os.listdir(TRAIN_DIR)[0]
    first_class_path = os.path.join(TRAIN_DIR, first_class)
    num_images = len([f for f in os.listdir(first_class_path) if f.endswith('.png')])
    print(f"Пример: в классе {first_class} найдено {num_images} изображений")
else:
    print("⚠ ВНИМАНИЕ: Папка Train не найдена! Проверьте путь.")

# %% [markdown]
# # 1. LOAD DATA FROM FOLDER STRUCTURE

# %%
print("Loading data from folder structure...")

# 2 параллельных пустых списка
train_paths = []
train_labels = []

# цикл по всем 43 классам, tqdm - прогресс бар
for class_id in tqdm(range(NUM_CLASSES), desc="Scanning class folders"):
    class_folder = os.path.join(TRAIN_DIR, str(class_id)) # собирает полный путь к
    папке текущего класса
    if os.path.exists(class_folder):
        for img_file in os.listdir(class_folder): # цикл по всем файлам в папке
            if img_file.endswith('.png'): # проверка формата файла
                img_path = os.path.join(class_folder, img_file) # собирает полный
                путь к текущему файлу
                train_paths.append(img_path) # добавляет полный путь в список
train_paths
                train_labels.append(class_id) # добавляет номер класса в список
train_labels

# сколько изображений нашлось
print(f"\nTotal training images found: {len(train_paths):,}")

# обработка ошибок
if len(train_paths) == 0:
    print("ERROR: No training images found! Please check the path:")
    print(f"TRAIN_DIR = {TRAIN_DIR}")

```

```

print("\nContents of Train directory:")
if os.path.exists(TRAIN_DIR):
    print(os.listdir(TRAIN_DIR)[:10])
else:
    print(f"Directory does not exist: {TRAIN_DIR}")
    raise Exception("No training images found")

class_counts = Counter(train_labels) # сколько раз встречается каждый класс
most_common = class_counts.most_common(1)[0] # класс с наибольшим колвом изображений
least_common = min(class_counts.items(), key=lambda x: x[1]) # класс с наим колвом
изображений, ищет минимум по второму элементу
print(f"Most common class      : {CLASS_NAMES[most_common[0]]} ({most_common[1]}
imgs)")
print(f"Least common class     : {CLASS_NAMES[least_common[0]]} ({least_common[1]}
imgs)")

# %% [markdown]
# # 2. EDA - VISUALIZE SAMPLES

# %%
# Exploratory Data Analysis (EDA) – разведочный анализ данных

# сортируем классы по убыванию количества изображений
sorted_counts = sorted(class_counts.items(), key=lambda x: x[1], reverse=True)

# создание окна с 2 графиками
fig, axes = plt.subplots(1, 2, figsize=(18, 7))
fig.suptitle('EDA - GTSRB Traffic Signs', fontsize=15, fontweight='bold')

# горизонтальная столбчатая диаграмма
colors = plt.cm.RdYlGn(np.linspace(0.2, 0.9, NUM_CLASSES))
axes[0].barh(range(NUM_CLASSES),
             [v for _, v in sorted_counts],
             color=colors, edgecolor='white', height=0.8)

# подписи на оси Y
axes[0].set_yticks(range(NUM_CLASSES))
axes[0].set_yticklabels([CLASS_NAMES[c][:22] for c, _ in sorted_counts], fontsize=6)

# заголовок, ось X, сетка
axes[0].set_title('Class Distribution (Training Set)')
axes[0].set_xlabel('Number of Images')
axes[0].grid(axis='x', alpha=0.3)

# примеры фотографий (5 классов по 3 фото)
sample_classes = [0, 1, 14, 17, 38]

img_grid = []
for cls in sample_classes:
    cls_indices = [i for i, label in enumerate(train_labels) if label == cls][:3]
    row_imgs = []

    for idx in cls_indices:
        # загружаем фото как тензор с помощью torchvision
        img_tensor = read_image(train_paths[idx])

        # Изменяем размер до 48x48 с помощью torchvision
        resize = T.Resize((48, 48), antialias=True)
        img_resized = resize(img_tensor)

        # Переводим в numpy для отображения
        img = img_resized.permute(1, 2, 0).numpy()

```

```

        row_imgs.append(img)

# Если изображений меньше 3 — добавляем чёрные квадраты
while len(row_imgs) < 3:
    row_imgs.append(np.zeros((48, 48, 3), dtype=np.uint8))

img_grid.append(row_imgs)

# Собираем сетку
if img_grid:
    grid_img = np.vstack([np.hstack(row) for row in img_grid])
    axes[1].imshow(grid_img)
    axes[1].set_title('Sample Signs: Speed20 | Speed30 | Stop | No Entry | Keep
Right\n(3 examples each)')
    axes[1].axis('off')

plt.tight_layout()
plt.savefig('eda_traffic.png', dpi=150, bbox_inches='tight')
print("\n[Saved] eda_traffic.png")

# %% [markdown]
# # 3. TRAIN/VALIDATION SPLIT#

# %%
train_paths, val_paths, train_labels, val_labels = train_test_split(
    train_paths, train_labels,
    test_size=0.2, # 20% данных отправляем в validation
    random_state=42,
    stratify=train_labels # сохранение пропорции классов в выборках
)

print(f"\nTrain samples: {len(train_paths):,}")
print(f"Validation samples: {len(val_paths):,}")

# %% [markdown]
# # 4. DATASET CLASS

# %%
# наследуем базовый класс dataset из pytorch
class GTSRBDataset(Dataset):
    def __init__(self, paths, labels, transform=None):
        self.paths = paths # список всех путей к картинкам
        self.labels = labels # номера классов
        self.transform = transform # аугментация + ресайз

    # что вернуть при запросе элемента с номером idx
    def __getitem__(self, idx):
        # загрузка изображения, тензор
        img = read_image(self.paths[idx]) # формат [3(rgb), Высота, Ширина], тип
uint8 (от 0 до 255)
        img = img.float() / 255.0 # перевод из uint в float32, нормализация к [0.0,
1.0]

        # применяем трансформации если они были заданы
        if self.transform:
            img = self.transform(img)

        return img, int(self.labels[idx]) # возвращает обработанное изображение с
номером класса

# колво изображений

```

```

def __len__(self):
    return len(self.paths)

print("\nCalculating dataset statistics for normalization...")
# временные трансформации для статистики
temp_transform = T.Compose([
    T.Resize((IMG_SIZE, IMG_SIZE)), # уменьшаем до 32x32
    T.ToDtype(torch.float32, scale=True), # переводим в float32 и нормализуем
])
# временный датасет и загрузчик
temp_dataset = GTSRBDataset(train_paths, train_labels, transform=temp_transform)
temp_loader = DataLoader(temp_dataset, batch_size=64, shuffle=False, num_workers=0)

# 3 нуля для каждого цветового канала (r g b)
mean = torch.zeros(3) # среднее значение (средний цвет)
std = torch.zeros(3) # стандартное отклонение (насколько сильно цвета отличаются от
среднего)
total_samples = 0 # счетчик

for images, _ in tqdm(temp_loader, desc="Calculating mean/std"):
    batch_samples = images.size(0) # колво картинок в батче
    images = images.view(batch_samples, images.size(1), -1) # переводим каждую
картинку в вектор
    mean += images.mean(2).sum(0) # считаем среднее по пикселям и суммируем по батчу
    std += images.std(2).sum(0) # считаем станд откл и суммируем
    total_samples += batch_samples

# делим на колво изображений
mean /= total_samples
std /= total_samples

print(f"Dataset mean: {mean}")
print(f"Dataset std: {std}")

# трансформация для обучения (аугментация)
train_transform = T.Compose([
    T.Resize((IMG_SIZE, IMG_SIZE)),
    T.RandomRotation(20),
    T.RandomAffine(degrees=0, translate=(0.15, 0.15)),
    T.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.3),
    T.ToDtype(torch.float32, scale=True),
])

# трансформация для проверки
val_transform = T.Compose([
    T.Resize((IMG_SIZE, IMG_SIZE)),
])

# два датасета: для обучения и для валидации
train_dataset = GTSRBDataset(train_paths, train_labels, transform=train_transform)
val_dataset = GTSRBDataset(val_paths, val_labels, transform=val_transform)
# создаем загрузчики, train перемешивает данные при обучении (shuffle)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
                           num_workers=0, pin_memory=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False,
                        num_workers=0, pin_memory=True)

# преобладают темные цвета, больше всего красного. цвета однородные, мало шума

# %% [markdown]
# # 5. CUSTOM CNN

```

```

# %%
class TrafficSignCNN(nn.Module):
    def __init__(self, num_classes=43):
        super().__init__()
        # сверточная часть (извлекает признаки)
        self.features = nn.Sequential(
            # входная картинка 32x32
            # если больше - модель точнее но медленнее, меньше - наоборот
            # с каждым слоем уменьшаем размер, увеличиваем колво каналов
            # сначала ищем простые признаки (края, линии), потом сложные (формы,
цифры)

            # слой 1
            # 3 - входные каналы тк rgb
            # 32 - выходные каналы, колво карт признаков
            # 3 - размер ядра свертки, окно скана(3x3), извлекает признаки
            # padding=1 - добавляем по 1пкс с каждой стороны, чтобы не терять края
            nn.Conv2d(3, 32, 3, padding=1),
            nn.BatchNorm2d(32), # нормализует значения в каждом канале
            nn.ReLU(inplace=True), # функция активации, обнуляет отрицательное
            nn.Conv2d(32, 32, 3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2), # окно 2x2 с шагом 2пкс, уменьшает размер в 2 раза,
оставляет важное
            # 16x16

            # слой 2
            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Conv2d(64, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),
            # 8x8

            # слой 3
            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Conv2d(128, 128, 3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(2, 2),
            # 4x4

            nn.Dropout2d(0.25), #выключает 25% рандомных каналов, против переобучения
        )

        # финальный слой, классификатор (принимает решение)
        self.classifier = nn.Sequential( # Sequential - контейнер
            # превращаем в один длинный вектор
            nn.Flatten(),
            # 1ый полносвязный слой, из 2048 нейронов оставляем 512
            nn.Linear(128 * 4 * 4, 512),
            # relu обнуляет отриц значения, нелинейная модель лучше обучается
            nn.ReLU(inplace=True),
            # выключает 50% нейронов, регуляризация
            nn.Dropout(0.5),
            # 2ой полносвязный, сжатие до 256 признаков
            nn.Linear(512, 256),

```

```

        nn.ReLU(inplace=True),
        nn.Dropout(0.3),
        # последний слой, выдает 43 числа (на каждый класс), потом преобразуем в
        # вероятности
        nn.Linear(256, num_classes),
    )

    # проход от входной картинки до 43 чисел
    def forward(self, x):
        x = self.features(x) # поиск признаков
        return self.classifier(x) # определение знака

model = TrafficSignCNN(NUM_CLASSES).to(DEVICE)
# считаем колво параметров
n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
# подсчет потерь (ошибок) модели
criterion = nn.CrossEntropyLoss()
# adaptive moment estimation (Adam) - универсальный оптимизатор, учитывает градиенты
# (инерцию)
# learning rate (LR) - базовая скорость обучения, уменьшает большие веса
# (регуляризация)
optimizer = optim.Adam(model.parameters(), lr=LR, weight_decay=1e-4)
# планировщик скорости обучения, если в течение 3 эпох loss не меняется - уменьшаем
# скорость
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5,
patience=3)

print(f"\nCustom CNN parameters: {n_params:,}")

# %% [markdown]
# # 6. TRAINING LOOP WITH TQDM

# %%

print("\n" + "=" * 55)
print("TRAINING CUSTOM CNN")
print("=" * 55)

history = {'train_loss':[], 'train_acc':[], 'val_loss':[], 'val_acc':[]} # словарь с
метриками каждой эпохи
best_val_acc, best_weights = 0.0, None

for epoch in range(1, EPOCHS + 1):
    model.train() # переводим модель в режим обучения
    run_loss, correct, total = 0.0, 0, 0

    train_pbar = tqdm(train_loader, desc=f'Epoch {epoch}/{EPOCHS} [Train]',
leave=False)
    for imgs, labels in train_pbar: # перебираем батчи
        imgs, labels = imgs.to(DEVICE), labels.to(DEVICE) # перемещает батч на
cpu/gpu

        optimizer.zero_grad() # обнуляет градиенты
        outputs = model(imgs) # прямой проход
        loss = criterion(outputs, labels) # считаем функцию потерь
        loss.backward() # обратное распространение ошибки
        optimizer.step() # обновляем веса с помощью оптимизатора

    # подсчет метрик
    run_loss += loss.item() * imgs.size(0)
    preds = outputs.argmax(1)
    correct += preds.eq(labels).sum().item()

```



```

        total += imgs.size(0)

        train_pbar.set_postfix({
            'loss': f'{loss.item():.4f}',
            'acc': f'{correct/total:.4f}'
        })

    t_loss = run_loss / total
    t_acc = correct / total

    model.eval() # переводим модель в режим оценки
    v_loss, v_correct, v_total = 0.0, 0, 0

    val_pbar = tqdm(val_loader, desc=f'Epoch {epoch}/{EPOCHS} [Val]', leave=False)
    with torch.no_grad(): # отключает вычисление градиентов
        for imgs, labels in val_pbar:
            imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
            outputs = model(imgs)
            loss = criterion(outputs, labels)

            v_loss += loss.item() * imgs.size(0)
            v_correct += outputs.argmax(1).eq(labels).sum().item()
            v_total += imgs.size(0)

            val_pbar.set_postfix({
                'loss': f'{loss.item():.4f}',
                'acc': f'{v_correct/v_total:.4f}'
            })

    v_loss = v_loss / v_total
    v_acc = v_correct / v_total

    scheduler.step(v_loss) # обновляем планировщик скорости обучения

    history['train_loss'].append(t_loss)
    history['train_acc'].append(t_acc)
    history['val_loss'].append(v_loss)
    history['val_acc'].append(v_acc)

    # сохраняем лучшую модель
    if v_acc > best_val_acc:
        best_val_acc = v_acc
        best_weights = {k: v.cpu().clone() for k, v in model.state_dict().items()}
        print(f"\nEpoch {epoch}: Train Acc={t_acc:.4f}, Val Acc={v_acc:.4f} (BEST)
√")
    else:
        print(f"\nEpoch {epoch}: Train Acc={t_acc:.4f}, Val Acc={v_acc:.4f}
(lr={optimizer.param_groups[0]['lr']:.6f})")

    # сохраняем лучшие веса
    if best_weights is not None:
        model.load_state_dict(best_weights)
        torch.save(model.state_dict(), 'best_traffic_cnn.pth')
        print(f"\nBest Validation Accuracy: {best_val_acc:.4f}
({best_val_acc*100:.2f}%)")
        print("[Saved] best_traffic_cnn.pth")
    else:
        print("\nWARNING: No best weights saved. Using final model.")

# %% [markdown]
# # 7. TRAINING CURVES

```

```

# %%
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Training History - Custom CNN', fontsize=14, fontweight='bold')

axes[0].plot(history['train_loss'], color='steelblue', linewidth=2, label='Train')
axes[0].plot(history['val_loss'], color='coral', linewidth=2, label='Val')
axes[0].set_title('Loss per Epoch')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].legend()
axes[0].grid(alpha=0.3)

axes[1].plot([a*100 for a in history['train_acc']], color='steelblue', linewidth=2,
label='Train')
axes[1].plot([a*100 for a in history['val_acc']], color='coral', linewidth=2,
label='Val')
if best_val_acc > 0:
    axes[1].axhline(best_val_acc*100, color='green', linestyle='--', alpha=0.7,
label=f'Best={best_val_acc*100:.2f}%')
axes[1].set_title('Accuracy per Epoch')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy (%)')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig('training_curves.png', dpi=150, bbox_inches='tight')
print("[Saved] training_curves.png")

# %% [markdown]
# # 8. CONFUSION MATRIX

# %%
model.eval()
all_preds, all_labels = [], []

print("\nGenerating predictions for confusion matrix...")
with torch.no_grad():
    for imgs, labels in tqdm(val_loader, desc="Predicting"):
        imgs = imgs.to(DEVICE)
        preds = model(imgs).argmax(1).cpu().numpy()
        all_preds.extend(preds)
        all_labels.extend(labels.numpy())

val_acc_final = np.mean(np.array(all_preds) == np.array(all_labels))
print(f"\nFinal Validation Accuracy: {val_acc_final:.4f} ({val_acc_final*100:.2f}%)")

fig, ax = plt.subplots(figsize=(16, 14))
cm = confusion_matrix(all_labels, all_preds, labels=range(NUM_CLASSES))
sns.heatmap(cm, cmap='Blues', ax=ax, linewidths=0.2, annot=False,
xticklabels=range(NUM_CLASSES), yticklabels=range(NUM_CLASSES))
ax.set_title(f'Confusion Matrix - Validation Set (Acc={val_acc_final:.4f})',
fontsize=14, fontweight='bold')
ax.set_xlabel('Predicted')
ax.set_ylabel('True')
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150, bbox_inches='tight')
print("[Saved] confusion_matrix.png")

# %% [markdown]
# # 9. TEST SET EVALUATION (using Test.csv)

```

```

# %%
print("\n" + "-" * 55)
print("Evaluating on Test Set")
print("-" * 55)

# Load test data from CSV
if os.path.exists(TEST_CSV):
    test_df = pd.read_csv(TEST_CSV)
    print(f"Test CSV shape: {test_df.shape}")
    print(f"Test CSV columns: {test_df.columns.tolist()}")

    test_paths = []
    test_labels = []

    for idx, row in tqdm(test_df.iterrows(), total=len(test_df), desc="Loading test
images"):
        possible_paths = [
            os.path.join(TEST_DIR, row['Path']),
            os.path.join(TEST_DIR, os.path.basename(row['Path'])),
        ]

        for img_path in possible_paths:
            if os.path.exists(img_path):
                test_paths.append(img_path)
                test_labels.append(row['ClassId'])
                break

    print(f"\nValid test images found: {len(test_paths):,}")

    if len(test_paths) > 0:
        test_dataset = GTSRBDataset(test_paths, test_labels, val_transform)
        test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False,
num_workers=0, pin_memory=False)

        model.eval()
        test_correct, test_total = 0, 0
        test_preds, test_true = [], []

        print("Evaluating on test set...")
        with torch.no_grad():
            for imgs, labels in tqdm(test_loader, desc="Testing"):
                imgs, labels = imgs.to(DEVICE), labels.to(DEVICE)
                outputs = model(imgs)
                preds = outputs.argmax(1)

                test_correct += preds.eq(labels).sum().item()
                test_total += imgs.size(0)

                test_preds.extend(preds.cpu().numpy())
                test_true.extend(labels.cpu().numpy())

        test_acc = test_correct / test_total
        print(f"\nTest Accuracy: {test_acc:.4f} ({test_acc*100:.2f}%)")

        print("\nClassification Report (Top 10 classes):")
        print(classification_report(test_true, test_preds,
labels=list(range(10)),
target_names=[CLASS_NAMES[i] for i in range(10)],
digits=3))
    else:
        print("No test images found. Skipping test evaluation.")

```

```

else:
    print(f"Test CSV not found at {TEST_CSV}. Skipping test evaluation.")

# %% [markdown]
# # 10. AUGMENTATION VISUALIZATION

# %%
if len(train_paths) > 0:
    aug_transform = T.Compose([
        T.ToPILImage(),
        T.RandomRotation(20),
        T.RandomAffine(degrees=0, translate=(0.15, 0.15)),
        T.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.3),
        T.ToTensor(),
    ])

    sample_tensor = read_image(train_paths[100])  # Tensor [3, H,
W]
    sample_img = sample_tensor.permute(1, 2, 0).numpy()  # для показа
matplotlib

    fig, axes_aug = plt.subplots(2, 5, figsize=(14, 6))
    fig.suptitle('Data Augmentation Examples (Bonus)', fontsize=13,
fontweight='bold')

    axes_aug[0, 0].imshow(sample_img)
    axes_aug[0, 0].set_title('Original', fontsize=9)
    axes_aug[0, 0].axis('off')

    for i in range(1, 10):
        aug = aug_transform(sample_tensor)  # подаём тензор
        aug = aug.permute(1, 2, 0).numpy()
        aug = np.clip(aug, 0, 1)
        r, c = divmod(i, 5)
        axes_aug[r, c].imshow(aug)
        axes_aug[r, c].set_title(f'Aug #{i}', fontsize=9)
        axes_aug[r, c].axis('off')

    plt.tight_layout()
    plt.savefig('augmentation_examples.png', dpi=150, bbox_inches='tight')
    print("[Saved] augmentation_examples.png")

# %% [markdown]
# # FINAL SUMMARY

# %%
print("\n" + "=" * 55)
print("FINAL SUMMARY")
print("=" * 55)
print(f"Dataset           : GTSRB")
print(f"Training samples    : {len(train_paths):,}")
print(f"Validation samples   : {len(val_paths):,}")
if 'test_paths' in locals():
    print(f"Test samples        : {len(test_paths):,}")
print(f"Classes              : {NUM_CLASSES}")
print()
print(f"Custom CNN (PyTorch, trained from scratch)")
print(f"  Input size         : {IMG_SIZE}x{IMG_SIZE}")
print(f"  Parameters          : {n_params:,}")
print(f"  Best Val Acc        : {best_val_acc*100:.2f}%")
if 'test_acc' in locals():
    print(f"  Test Acc            : {test_acc*100:.2f}%")

```

```
print()
print("Output files saved:")
print("  eda_traffic.png | training_curves.png | confusion_matrix.png")
print("  augmentation_examples.png")
if best_weights is not None:
    print("  best_traffic_cnn.pth")
```